

ECHO: Efficient Covertly-Secure Three-party Computation with Applications to Private Machine Learning

Yufei Duan, Yun Li, Zhicong Huang, Cheng Hong, Tao Wei, and Chao Zhang

Abstract—Secure three-party computation with an honest majority is one of the most efficient settings in secure computation, and has been widely adopted in practical applications. However, achieving malicious security in this setting incurs significant concrete efficiency penalties, which could be an order of magnitude worse than that of their semi-honest counterparts. Covert security offers a potential security-efficiency trade-off by detecting malicious behavior with a certain probability (such as 50%), deterring rational adversaries through the risk of detection and loss of credibility. Yet, existing covert security research primarily focuses on two-party or general n -party protocols in the dishonest-majority setting, with limited progress toward efficient three-party solutions.

This work presents the first comprehensive framework, ECHO, for covertly secure, honest-majority three-party computation with applications to privacy-preserving machine learning. We systematically explore the design space of cheating detection and cheater identification techniques, and propose a suite of novel protocols for both arithmetic and Boolean circuits. Each protocol is engineered for a distinct performance goal: minimal online latency, high end-to-end efficiency, or low communication. Notably, for arithmetic circuits over rings, we introduce a protocol leveraging asymmetric message authentication codes, achieving an online phase that is only $1.26\times$ slower than the semi-honest baseline, over three times faster than its maliciously secure counterpart. For Boolean circuits, our novel cut-and-choose-based method outperforms the best previous maliciously secure protocol by a factor of five. In practical PPML benchmarks, our framework achieves near semi-honest performance while delivering up to $8\times$ speedup over maliciously secure protocols on real-world tasks.

Index Terms—Secure multi-party computation, three-party computation, covert security, privacy-preserving machine learning, honest majority.

I. INTRODUCTION

Secure Multiparty Computation (MPC), introduced by Andrew C. Yao in 1982 [65], enables mutually distrustful parties to jointly compute a function on their private inputs while revealing only the output. In recent years, MPC has evolved from a purely theoretical construct to a practical technology, examples include private machine learning [54], [19], [56], [46], statistics [64], or voting [57].

The threat models of MPC can be classified into two types: **semi-honest security**, where adversaries follow the protocol but attempt to infer additional information, and **malicious security**, where adversaries may cheat and arbitrarily deviate

from the protocol. Recent works [47], [42] have shown that malicious parties in MPC can compromise both data integrity and privacy, emphasizing the need for malicious security. However, current maliciously secure solutions impose significant performance penalties. For example, SOTA works [54], [26], [49], [48] utilize secret MACs (Message Authentication Codes), DZKPs (Distributed Zero-Knowledge Proofs), or Beaver triples [8] to guarantee malicious security, but they are $3 \sim 83\times$ slower compared to their semi-honest counterparts as we benchmarked.

In 2007, Aumann and Lindell [7] introduced **covert security** as an intermediate threat model to balance security and efficiency. Unlike malicious security's overwhelming anomaly detection guarantee, covert security provides probabilistic detection with deterrence factor ϵ thus reducing the verification overhead. For rational business parties, this security model is already powerful enough, because the possibility of public exposure and reputation loss will discourage malicious behaviors, especially since mounting a successful attack often necessitates multiple rounds of cheating without being caught [47].

However, there still exist significant gaps in designing concrete efficient covert protocols. Most prior works on covert security focus on two-party (2PC) [7], [5], [45], [38] or general n -party computation (n -PC) [31], [62] with a dishonest majority, while little attention has been paid to the three-party (3PC) honest-majority setting, which enables concretely efficient protocols and has been widely studied in the applied MPC literature [3], [32], [2], [12], [48], [46], [56]. However, this gap is further complicated by the critical requirement of **cheater identification**:

- Even maliciously secure protocols don't inherently identify cheaters, because the honest party can simply abort¹ upon detecting any abnormality. In contrast, covert security fundamentally requires (publicly verifiable) cheater identification. Merely detecting an anomaly is insufficient for deterrence; the risk of exposure and reputational damage is central to the covert security model.
- Unlike in 2PC, where any cheating detection directly reveals the cheating party, honest parties in 3PC face ambiguity when attempting to identify which of the remaining two parties is the cheater.
- While [62], [31] propose n -party covert protocols with cheater identification, they are restricted to input-independent preprocessing protocols or are costly to adapt

Y. Duan and C. Zhang are with the Institute for Network Sciences and Cyberspace, BNRist, Tsinghua University, Beijing 100084, China (e-mail: dyf23@mails.tsinghua.edu.cn; chaoz@tsinghua.edu.cn).

Y. Li, Z. Huang, C. Hong, and T. Wei are with Ant Group, Hangzhou 310000, China (e-mail: liyun24@antgroup.com; zhicong.hzc@antgroup.com; vince.hc@antgroup.com; lenx.wei@antgroup.com).

Manuscript received xxx; revised xxx.

¹Protocols like [46] ask all the parties to send inputs to the honest party to compute the function upon failure. While effective for guaranteed output delivery (G.O.D.), this approach still does not support cheater identification.

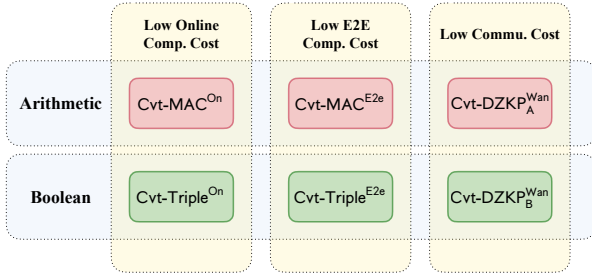


Fig. 1: Overview of our covertly secure 3PC protocols and their properties. (“E2E”: end-to-end, “Comp.”: computational, “Commu.”: communication.)

to input-dependent protocols, and it’s unexplored how to apply their protocols to practical MPC scenarios.

Consequently, no efficient covertly secure 3PC frameworks currently exist, and addressing this gap presents substantial technical challenges.

A. Problem Definition and Threat Model

The data (e.g., model parameters and images) is secret shared among three parties. They want to jointly run a function (e.g., apply a neural network inference) on the data to obtain an output (e.g., inference result), without letting any party know anything beyond the output and public information (e.g., hyperparameters such as model structure and image size). The function is typically expressed as an arithmetic or a Boolean circuit. As rings like $\mathbb{Z}_{2^k}$ are more commonly adopted than fields (especially in PPML), in this work we mainly focus on arithmetic circuits over $\mathbb{Z}_{2^k}$ and Boolean circuits over $\mathbb{Z}_2$.

We assume only one party will be corrupted and try to run arbitrary malicious behavior to gain advantages. Our protocols aim to detect such malicious behavior with (at least) a given probability ϵ , and identify the malicious party after detecting such behavior; in addition, we allow the honest parties to generate a publicly verifiable certificate that can be validated by any third party to confirm the dishonesty of the cheater.

B. Technical Contributions

In this work, we present ECHO, a framework of covertly secure 3PC protocols with public verifiability for arithmetic circuits and Boolean circuits. We give an overview of our protocols in Figure 1, and compare our covertly secure protocols with existing SOTA 3PC protocols in the semi-honest and malicious model in communication cost in Table I.

At a high level, covert security can be achieved via: (1) detecting cheating behavior, then (2) identifying the cheater (with public verifiability). Below we first present our cheating detection approaches for $\mathbb{Z}_{2^k}$ and $\mathbb{Z}_2$ respectively, and then show how to identify the cheater with public verifiability.

1) *Cheating Detection over $\mathbb{Z}_{2^k}$* : We design three protocols to detect cheating behavior over $\mathbb{Z}_{2^k}$: the first two adapt existing MAC-based and DZKP-based cheating detection techniques using smaller ring gaps, and the third one is specially designed for low online cost.

TABLE I: Communication cost of different honest-majority 3PC protocols for arithmetic circuits over $\mathbb{Z}_{2^k}$ and k -bit parallel Boolean circuits over $\mathbb{Z}_2$. $\epsilon < 1$ is the deterrence factor. s and s^* denote the ring gap in malicious and covert security, respectively. Typically s is set to 46 and 64 in [1] and [49], and s^* is set to 3 ~ 12 in ECHO. Communication costs are global. All costs are amortized over gates.

Protocol	Threat Model	Offline Commu.	Online Commu.
[3]	Semi-honest	-	$3k$
[19]		k	$2k$
[1]		-	$6(k + s)$
[49], [48]	Malicious	-	$3k$
[32], [43]		$21k$	$6k$
[2]		$12k$	$9k$
[23], [43] [†]		$9(k + s)$	$6k$
Cvt-MAC ^{E2e}	Covert ($\epsilon = 50\%$)	-	$6(k + s^*)$
Cvt-MAC ^{On}		$2(k + s^*)$	$3(k + s^*)$
Cvt-DZKP ^{Wan} _A		-	$3k$
Cvt-DZKP ^{Wan} _B		-	$3k$
Cvt-Triple ^{On}		$6k$	$6k$
Cvt-Triple ^{E2e}		-	$7.5k$

([†]): The authors of MP-SPDZ [43] adapts the techniques from [23] to the three-party honest-majority setting.

(1) **Cvt-MAC^{E2e}, Cvt-DZKP^{Wan}_A: Adapting MACs and DZKPs.** A key insight is that while the “ring gap” required for malicious security is moderate, it necessitates multi-precision arithmetic that incurs a drastic performance penalty. Specifically, for arithmetic circuits over the ring $\mathbb{Z}_{2^k}$, the two SOTA cheating detection techniques, MACs [1] and DZKPs [49], both perform the verification over a sufficiently large ring $\mathbb{Z}_{2^{k+s}}$, where, for example, $s = 46$ and 64 respectively for ensuring 2^{-40} soundness error. For $k = 64$, the ring will be lifted to $\mathbb{Z}_{2^{110}}$ and $\mathbb{Z}_{2^{128}}$, respectively. However, if we bound the soundness guarantee to 50%, the ring gap s will decrease to 3 and 12; for most applications, this allows us to embed the ring gap s into $\mathbb{Z}_{2^{64}}$, (i.e., set $k = 61$ and 52 respectively s.t. $k + s = 64$). Given the fact that multiplications over $\mathbb{Z}_{2^{64}}$ are $2 \sim 3\times$ faster than those over $\mathbb{Z}_{2^{110}}$ and $\mathbb{Z}_{2^{128}}$, this directly leads to a notable efficiency gain.

We denote the above two approaches by Cvt-MAC^{E2e} and Cvt-DZKP^{Wan}_A, which inherently feature a high end-to-end computational efficiency and sublinear communication cost (that outperforms in WAN), respectively.

(2) **Cvt-MAC^{On}: Achieving a Lightweight Online Phase.** We notice that the verification overheads of the above MAC-based [1] and DZKP-based [49] approach are incurred online; however, one may expect a more efficient online phase if they can perform preprocessing. On the other hand, ASTRA [19] adopts an asymmetric online-efficient setting where P_0 distributes correlated randomness in the preprocessing phase, and P_1, P_2 evaluate the circuit in the online phase, but its triple-based cheating detection approach entails high offline communication costs.

We design a new online-efficient cheating detection approach by exploiting the asymmetric paradigm of ASTRA but using MACs. We observe that the correlated randomness generated by P_0 can be verified in the preprocessing phase, and

if all the checks are passed, we can let P_0 sample the random MAC key (instead of the parties together generating a shared one as in the original MAC-based protocol [1]). With P_0 offering additional correlated randomness in the preprocessing phase, P_1, P_2 can *locally* authenticate each multiplication output in the online phase, and thus incur no extra (amortized) online communication cost over the semi-honest protocol of ASTRA (except that caused by the expansion of the ring). As a result, we can reduce the online communication cost from $6(k+s)$ bits per gate in [1] to $2(k+s)$ bits, with $3(k+s)$ offline communication bits.

We emphasize that this new approach may be of independent interest, e.g., it can be used for constructing maliciously secure protocols with all the asymptotic improvements maintained. In the covert model, we achieve ϵ deterrence factor by controlling the ring gap s as above. We denote the new online-efficient cheat detection approach by $\text{Cvt-MAC}^{\text{On}}$.

2) *Cheating Detection over $\mathbb{Z}_2$* : We also design three protocols for Boolean circuits: the first one adapts the DZKP-based approach [9], [12], and the latter two are based on triples with a specially designed cut-and-choose paradigm.

(1) $\text{Cvt-DZKP}_B^{\text{Wan}}$: **Adapting DZKPs**. When verifying semi-honest behaviors over $\mathbb{Z}_2$, we need to lift the verification onto a large binary extension field to achieve the given soundness guarantee. Thus, in the covert model we can control the size of the extension field to achieve the specific deterrence factor.

(2) $\text{Cvt-Triple}^{\text{E2e}}, \text{Cvt-Triple}^{\text{On}}$: **A New Cut-and-Choose Paradigm in the Covert Model**. In SOTA triple-based cheating detection approaches [32], [2], [43], the parties typically generate random AND triples using the cut-and-choose paradigm in the preprocessing phase, and then verify semi-honest online AND triples using these random offline triples. Intuitively, in the covert model we can loosen the soundness guarantee by adapting the involved parameters, e.g., using a smaller bucket size B . However, we notice that for slightly large circuit sizes (e.g., 100) and moderate deterrence factors (e.g., 50% or 75%), even the smallest bucket size $B = 2$ will lead to exceeded soundness guarantees with unnecessary overheads. Therefore, we tailor a new cut-and-choose paradigm for the covert model, by allowing the parties to output some unchecked triples. Specifically, we let the parties generate fewer initial triples, and only check a random part of them; ultimately, all those unchecked triples will be output along with those checked ones. This enables us to significantly reduce the offline overheads by adjusting the number of initial triples and the proportion of the checked triples properly.

We design two variants of the above approach; one features a higher online efficiency, and the other features a higher end-to-end computational efficiency. We denote them by $\text{Cvt-Triple}^{\text{On}}$ and $\text{Cvt-Triple}^{\text{E2e}}$, respectively.

3) *Cheater Identification with Public Verifiability*: We note that for the DZKP-based approaches ($\text{Cvt-DZKP}_A^{\text{Wan}}$ and $\text{Cvt-DZKP}_B^{\text{Wan}}$) and the one working in the asymmetric preprocessing setting ($\text{Cvt-MAC}^{\text{On}}$), cheater identification is almost free. For the others, we design a general cheater identification mechanism by using Commit-and-Prove Non-Interactive Zero-Knowledge (CP-NIZK) proofs.

(1) **A Lightweight Cheater Identification Mechanism**. We follow previous works [31], [62] using a signature and commitment scheme to make the parties accountable for their inputs and messages. We observe that in the three-party honest-majority setting, the key to identifying cheaters with almost free costs is to identify an honest party, who can act as a judge and identify the cheater by checking the other parties' views.

For the DZKP-based approaches, the core property to help identify an honest party is "re-computable verification" [12]. That is, an honest prover can always tell which verifier is cheating by re-computing the verifiers' messages. Therefore, once an abortion occurs, we can let the prover accuse one verifier, and the other verifier will be viewed as honest. For the asymmetric MAC-based approach, the re-computable verification property also holds for P_0 . If the verification on P_0 is passed, then P_0 can be identified as an honest party; otherwise, we let P_0 accuse one verifier, and identify the left one party as honest. The above identification mechanisms introduce no heavy cryptographic primitives, and thus are efficient.

(2) **A General Cheater Identification Mechanism**. For the symmetric MAC-based and triple-based approaches, our idea is to enforce the parties to generate a zero-knowledge proof showing their honest behaviors after an abortion happens, i.e., proving that all its sent messages are faithfully computed according to the protocol specification w.r.t. its input, random tape and all the received messages; then the one who fails to provide a valid proof will be accused as the cheater.

(3) **Achieving Public Verifiability**. We observe that the three-party honest-majority setting is especially advantageous for achieving public verifiability, compared to the previous two-party or n -party dishonest-majority settings [31], [62], [38], [45]. In the three-party honest-majority setting, public verifiability follows naturally after identifying the cheater: the two honest parties can generate and sign a message that accuses the cheater, and then collect their signed message as a certificate. After that, any third party can confirm the dishonesty of the cheater by checking the two valid signed messages.

4) *Implementation and Evaluations*: We implement all protocols in MP-SPDZ [43]. Take deterrence factor $\epsilon = 50\%$ for example, in arithmetic benchmarks, $\text{Cvt-MAC}^{\text{E2e}}$ achieves $2.38\times$ end-to-end speedup over the best maliciously secure MAC-based method [1]; $\text{Cvt-DZKP}_A^{\text{Wan}}$ is $2.27\times$ faster than the best maliciously secure DZKP-based method [49]; more importantly, $\text{Cvt-MAC}^{\text{On}}$ achieves similar online costs as the semi-honest baselines, and is $3.56\times$ and $5.54\times$ faster than the SOTA triple-based maliciously secure method [23] in the online and offline phase, respectively.

In Boolean benchmarks, $\text{Cvt-DZKP}_B^{\text{Wan}}$ is $1.73\times$ faster than the best maliciously secure DZKP-based method [48]; $\text{Cvt-Triple}^{\text{E2e}}$ delivers $5.36\times$ end-to-end improvement over the best maliciously secure triple-based method [2]; $\text{Cvt-Triple}^{\text{On}}$ matches the online cost of the best online-oriented maliciously secure method [32], [43] while reducing the offline cost by $2.89\times$.

II. RELATED WORKS

A. Maliciously Secure Protocols

Below we review SOTA cheating detection approaches from existing maliciously secure MPC protocols.

MAC-based Approach: At a high level, in MAC-based approaches [1], [22], the parties authenticate the inputs and all outputs of multiplication gates using a random secret MAC key. For finite fields, the randomness and secrecy of the MAC key guarantee that the probability of a malicious adversary cheating successfully is negligible provided that the field is sufficiently large. However, unlike fields, directly using MACs over $\mathbb{Z}_{2^k}$ would entail large soundness error (i.e., $1/2$ in the worst case). Therefore, the previous work [1] proposed to perform the computation over a sufficiently larger ring $\mathbb{Z}_{2^{k+s}}$, which would reduce the soundness error to $2^{-s+\log(s+1)}$. However, as all the multiplication gates should be authenticated additionally, this approach incurs a (at least) doubled communication cost of $2(k+s)$ bits per gate per party (compared to k bits of the semi-honest protocol).

DZKP-based Approach: This approach [49] is derived from distributed zero-knowledge proofs introduced by [9] and then developed in [12], [13], [49], [14], [37]. At a high level, it applies to secret-share-based semi-honest protocols where all involved data is distributed among the parties, and verifies the semi-honest executions of the parties in a distributed way. A highlighting feature of this approach is that it only incurs a logarithmic communication cost. However, it requires a sufficiently large finite field to ensure a negligible soundness error. When working over the ring $\mathbb{Z}_{2^k}$, the SOTA work [49] proposes to combine the idea of the ring gap from [1] and the recursive techniques from [9], and achieves logarithmic communication cost. However, compared to the MAC-based approach, its computational cost is still relatively higher.

The DZKP-based approach also applies to Boolean circuits, but typically requires lifting the verification onto a sufficiently large binary extension field [9], [12] to guarantee negligible soundness errors. A recent work [48] proposed to alternatively lift the verification onto a prime field. By designing an efficient transformation method and applying a 61-bit Mersenne field, they can achieve higher concrete efficiency.

Triple-based Approach: Triple is another classic technique used for achieving malicious security for Boolean circuits [55], [27], [16], [32], [2], [19], based on the observation that any incorrect AND triple can be detected using a validated random AND triple with probability 100%. To efficiently generate validated AND triples, the previous work [32] adopted the cut-and-choose paradigm, where the parties generate and shuffle some initial triples, and check them via an open-then-bucket-check procedure (see details in Section V). For circuit size $N = 2^{20}$ and 40-bit soundness security, the protocol costs 7 bits to generate a validated triple, and 3 bits to compute and verify an online AND gate. The follow-up work [2] improves the approach of [32] by taking a smaller bucket size but adding an extra shuffle process on the online AND triples. This reduces the communication cost from 10 bits to 7 bits per gate per party. However, the online computational cost is slightly

heavier. In MP-SPDZ framework [43], the authors optimized the online phase of [32] by letting the parties directly compute an AND gate using a validated AND triple. In this way, the online communication cost is reduced to 2 bits per gate.

B. Covertly Secure Protocols

In the literature [7], [5], [45], [38], [28], [62], [31], covert security is typically achieved using the cut-and-choose paradigm [20] by running t instances of the semi-honest protocol in parallel and randomly opening $t-1$ of them for verification. This provides a deterrence factor of $1 - \frac{1}{t}$, however, incurring a t times overhead of the semi-honest protocol. In recent works [31], [62], public verifiability is further achieved using time-lock puzzles [59]. However, the paradigm only applies to input-independent preprocessing protocols due to privacy requirements, while the most efficient RSS-based 3PC protocol [3] doesn't fall into this category. To compile the 3PC protocol [3] into a covertly secure one, other techniques such as the player visualization paradigm [40], [51] are required, which would incur an explosion in communication and computational overhead. We give more details in Appendix C.

III. PRELIMINARIES

A. Notation

Let $\mathcal{P} = \{P_0, P_1, P_2\}$ be a set of three parties. We use P_j where $j \in \{0, 1, 2\}$ to denote a certain party in this set, and $P_{j\pm 1}$ to denote the next (+) or previous (−) party. We use $\mathbb{Z}_2$ for the binary ring of modulus 2, and $\mathbb{Z}_{2^k}$ for a ring of modulus 2^k . We may explicitly use $\equiv_k$ for congruence modulo 2^k , and use $:=$ for assignment. For simplicity, we use $[n]$ to denote the set $\{1, \dots, n\}$, and $[n_1, n_2]$ for $\{n_1, \dots, n_2\}$. We use $\vec{v}$ for a vector.

Let κ be the computational security parameter and σ the statistical security parameter. We use $\text{negl}(\cdot)$ to denote a negligible function. We use H for a pre-image and collision-resistant hash function and PRG for a pseudo-random generator with a key $K \in \{0, 1\}^\kappa$ (the target domain will be clear from context). We will use some shared keys to generate shared randomness, including a global key K known by all the parties, and another two keys K_t for $t \in \{1, 2\}$ known by P_0 and P_t , respectively.

B. Secure Multiparty Computation

We follow the standard ideal/real simulation paradigm, and use the covert notions from previous works [7], [5], [62], [31].

Covert Security with Public Verifiability. We consider the strongest security notion in the covert model, i.e., *strong explicit cheat formulation* (SECF). The ideal adversary can send a special message (cheat, i) to the ideal functionality, and $\mathcal{F}_{\text{SECF}}$ allows the cheating happens with probability $1 - \epsilon$; in this case, the ideal adversary $\mathcal{S}$ can learn the honest parties' inputs, and decide the output of the functionality. Covert security with public verifiability property further enhances a covertly secure protocol with two algorithms *Blame* and *Judge*. The algorithm *Blame* is used for an honest party to distill a certificate cert_j from its view after it outputs a cheater

P_i , denoted by $\text{cert}_j = \text{Blame}(\text{view}_j)$. The algorithm Judge can be executed by any third party, which takes as input an honest party's certificate cert_j and outputs the identity of a cheating party, i.e., $\text{id}_i = \text{Judge}(\text{cert}_j)$. We give a detailed description of the definition in Appendix A.

Security up to Additive Attacks. In this work we consider security up to additive attacks [35], [34], [22], [1], [49], [48], i.e., the only way an adversary can cheat when computing a multiplication gate is to add a non-zero error to the result. It captures the property of many existing semi-honest multiplication protocols [3], [19], [63], including the ones we use in this work.

C. Sharing Semantics

Additive Secret Sharing (ASS): A value x is said to be additively secret-shared between parties P_1, P_2 , if each party P_j for $j \in \{1, 2\}$ hold a random additive share x_j s.t. $x = x_1 + x_2$. We denote it by $[x] = (x_1, x_2)$.

Replicated Secret Sharing (RSS) [41]: A value x is said to be replicated-secret-shared among parties P_0, P_1, P_2 , if each party P_j for $j \in \{0, 1, 2\}$ holds two random additive shares x_j, x_{j-1} s.t. $x = x_0 + x_1 + x_2$. We denote it by $[x] = (x_0, x_1, x_2)$.

We may use superscripts to specify the underlying domain of this share, i.e., $[\cdot]^A, [\cdot]^B$ denote RSS over the arithmetic ring $\mathbb{Z}_{2^k}$ and the binary ring $\mathbb{Z}_2$ respectively.

Pairwise Consistency. We say $[x]$ is pairwise consistent if the additive share that is commonly held by the two honest parties is consistent. We adopt a procedure “cmpview” from [32] for the parties to check their common shares. Specifically, for each pair of two parties P_j, P_{j+1} holding a common additive share x_j , we let P_j send a hash digest $H(x_j)$ to P_{j+1} , who then checks if the digest is consistent with the share x_j held by itself. If any inconsistency is detected, the parties will abort.

Reconstruction. To reconstruct a secret x to a party P_j , we let P_{j+1} send x_{j+1} to P_j , who then sums the three shares up to recover x . To open x to all the parties, we simply run the above procedure for each $j \in \{0, 1, 2\}$. However, a malicious party may send an incorrect share during reconstruction. We use cmpview to detect such behaviors: since P_{j+1} and P_{j-1} both hold x_{j+1} , we additionally let P_{j-1} send a hash digest $H(x_{j+1})$ to P_j , who then will check consistency of the received x_{j+1} and $H(x_{j+1})$. We denote the above procedure by Rec.

Linearity. RSS is linearly homomorphic, and thus addition and multiplication-by-constant can be locally computed by the parties. Specifically, given two sharings $[x], [y]$, and any public values a, b , they can locally compute $[z] := [ax + by]$ by each party P_j setting $(z_{j-1}, z_j) := (ax_{j-1} + by_{j-1}, ax_j + by_j)$. For any public constant c , obtaining $[x + c]$ only requires any two parties adding c to their common additive share of x .

Multiplication. To compute $[z]$ where $z = x \cdot y$ given $[x], [y]$ (which is denoted by $[x] \cdot [y]$ in this work), first note that each P_j can locally compute an additive share of z , i.e., $z_j :=$

Protocol 3.1: Π_{Mul} (Multiplication in ASTRA Sharing)

Input: $[x] = (r_{x,1}, \delta_x, r_{x,2})$ and $[y] = (r_{y,1}, \delta_y, r_{y,2})$.

Preprocessing:

- 1) P_0 computes $r_{xy} := r_x \cdot r_y$ where $r_x = r_{x,1} + r_{x,2}$, $r_y = r_{y,1} + r_{y,2}$, and then shares r_{xy} to P_1, P_2 .
- 2) P_0 and P_t for $t \in \{1, 2\}$ run $r_{z,t} := \text{PRG}_{K_t}(\cdot)$. P_0 sets its share of z as $(r_{z,1}, r_{z,2})$.

Online:

- 1) P_1 computes $\delta_{z,1} := \delta_x \delta_y + \delta_x r_{y,1} + \delta_y r_{x,1} + r_{xy,1} - r_{z,1}$. P_2 computes $\delta_{z,2} := \delta_x r_{y,2} + \delta_y r_{x,2} + r_{xy,2} - r_{z,2}$. P_1, P_2 exchange $\delta_{z,1}, \delta_{z,2}$ with each other.
- 2) P_1, P_2 locally compute $\delta_z := \delta_{z,1} + \delta_{z,2}$, and set their share as $(\delta_z, r_{z,1})$ and $(\delta_z, r_{z,2})$ respectively.

$x_j y_j + x_j y_{j-1} + x_{j-1} y_j$ s.t. $z = z_0 + z_1 + z_2$. To convert it into a replicated secret sharing, each P_j needs to pass around its share z_j to the next party, and to guarantee privacy, z_j should be masked with an additive share of 0, which can be locally generated using PRG with shared keys [41].

ASTRA Secret Sharing: We introduce a variant of the RSS scheme from ASTRA [19]. In this scheme, a secret x is masked by a randomness r_x s.t. $x = \delta_x + r_x$, and

- δ_x is held by P_1, P_2 ,
- r_x is held by P_0 , and is additively shared between P_1, P_2 .

Concretely, P_0 holds randomness $(r_{x,1}, r_{x,2})$ s.t. $r_x = r_{x,1} + r_{x,2}$, and P_1, P_2 hold $(\delta_x, r_{x,1})$ and $(\delta_x, r_{x,2})$, respectively. It's clearly the form of RSS, and we can denote it by $[x] = (r_{x,1}, \delta_x, r_{x,2})$.

ASTRA sharing inherits the linearity property from the RSS scheme. Pairwise consistency and the reconstruction procedure also follow naturally.

Multiplication. The multiplication protocol of ASTRA [19] slightly differs from that of the RSS scheme. For each multiplication gate computing $[z] := [x] \cdot [y]$, we have $z = (\delta_x + r_x)(\delta_y + r_y) = \delta_x \delta_y + \delta_x r_y + \delta_y r_x + r_x r_y$. As P_1, P_2 already hold δ_x, δ_y and $[r_x], [r_y]$, we let P_0 first provide an additive sharing of $r_{xy} := r_x r_y$ to P_1, P_2 (which can be done in the preprocessing phase); then P_1, P_2 can compute an additive sharing $[z] := \delta_x \delta_y + \delta_x [r_y] + \delta_y [r_x] + [r_{xy}]$ in the online phase. By letting P_0 generate a shared random mask $[r_z] = (r_{z,1}, r_{z,2})$ with P_1, P_2 (in the preprocessing phase), P_1, P_2 can mask $[z]$ by computing $[\delta_z] := [z] - [r_z]$. Then P_1, P_2 recover δ_z , and the parties can obtain a replicated secret sharing of z by setting $[z] := (r_{z,1}, \delta_z, r_{z,2})$. We present the protocol in Protocol 3.1.

D. Ideal Functionalities

We use some basic functionalities as our building blocks.

- $\mathcal{F}_{\text{rand}}$: This functionality samples a random r , and distributes a sharing $[r]$ to the parties. It can be instantiated non-interactively using PRG and shared keys.

- $\mathcal{F}_{\text{coin}}$: This functionality samples a random r , and sends the public r to the parties. It can be instantiated by first calling $\mathcal{F}_{\text{rand}}$ to obtain $\llbracket r \rrbracket$ then opening it.
- $\mathcal{F}_{\text{checkzero}}$: This functionality checks if an additively shared value equals 0 over a ring $\mathbb{Z}_{2^k}$. It receives two shares x_1, x_2 from two parties P_1, P_2 , and checks if $x_1 + x_2 =_k 0$. We can use the instantiation from [1].
- $\mathcal{F}_{\text{input}}$: This functionality allows a party to share its input x to the other parties. We can follow previous works [3], [19] to instantiate it. In Appendix D we give the instantiation of $\mathcal{F}_{\text{input}}$ with the ASTRA sharing scheme.
- $\mathcal{F}_{\text{VrfyIP}}$: This functionality is formally introduced by [12], [13], [49], which checks inner-product relations that are secret-shared among the parties and are known by a single party. The single party knowing the secrets, say P_j , acts as a prover, while the other two parties P_{j-1}, P_{j+1} holding the shares act as verifiers. We review the formal description and instantiation of the functionality from [49] in Appendix D. In this work, we will use the adapted functionality $\mathcal{F}_{\text{VrfyIP}}^\epsilon$ in the covert model, which detects incorrect distributed relations with probability ϵ .

E. Signature Schemes

We use a signature scheme that is existentially unforgeable under chosen-message attacks. We give a formal description of the signature scheme in Appendix B due to page limit.

F. CP-NIZK Proof Schemes

A commitment scheme [15] is a tuple of three algorithms $\text{COM} = (\text{Gen}, \text{Commit}, \text{Open})$ that allows a party to first commit a secret message then opening it (with some auxiliary information). It should be *binding* and *hiding*. The binding property guarantees that it's hard for an adversary to find two different messages with the same commitment, and the hiding property ensures that the commitment doesn't reveal any information about the message. We give a formal definition of commitments in Appendix B-B.

Let R be a polynomial-time decidable relation on the pair (s, w) , where s is an NP statement, and w is the witness. We write $R(s, w) = 1$ if relation R holds on (s, w) , and 0 otherwise. A Non-Interactive Zero-Knowledge (NIZK) proof of knowledge is a tuple of algorithms $\text{ZK} = (\text{Gen}, \text{Prove}, \text{Vrfy})$ that allows a party to prove the satisfaction of a relation R on a public statement s and secret witness w , without revealing any information about w . It should have the properties of *completeness*, *knowledge soundness*, and *zero-knowledge*. Completeness guarantees that an honest verifier will always accept a proof generated by an honest prover on a satisfied relation; knowledge soundness guarantees that a malicious prover cannot fake a proof to convince the verifier if the relation is not satisfied with w ; zero-knowledge ensures that no extra information is revealed other than the truth of the statement. Formal definitions can be found in Appendix B-B.

We adopt the notion of *Commit-and-Prove* NIZK (CP-NIZK) proofs of knowledge from LegoSNARK [18], [50].

Given a commitment scheme COM and a relation R , a CP-NIZK proof shows the knowledge of a secret witness w for a public statement s s.t. $R(s, w) = 1$, and additionally, $w = (u, \text{aux})$ can be split into a message part u and an auxiliary part aux , and the secret message is in accord with a specified commitment. $\text{Com}_u = \text{COM.Commit}(\text{pp}; u)$. More specifically, it demonstrates the following conjunctive relations: $\text{COM.Open}(\text{pp}; \text{Com}_u, u, r) = 1 \wedge R(s, (u, \text{aux})) = 1$. Existing works [52], [6], [17], [4], [53], [33] have well explored this type of proof system, and can be adopted in this work.

IV. CHEATING DETECTION ON ARITHMETIC CIRCUITS

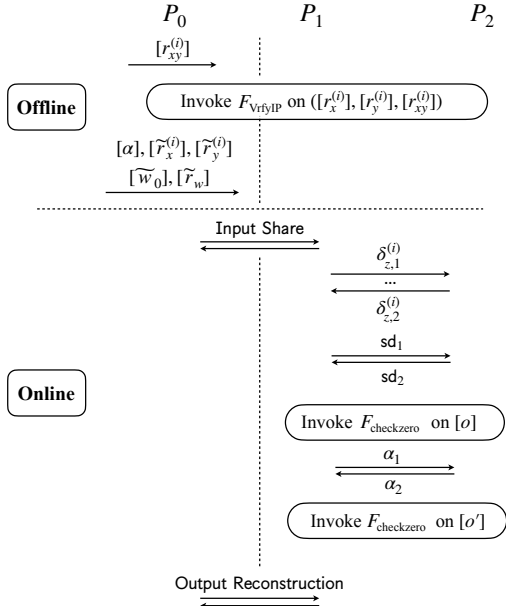
In this section, we present three cheating detection approaches for arithmetic circuits over $\mathbb{Z}_{2^k}$: the first two are based on smaller ring gaps with high end-to-end efficiency or low communication cost, and the last one focuses on online efficiency.

A. Approaches Based on Smaller Ring Gaps

For SOTA maliciously secure approaches based on MACs [1] and DZKPs [49], they need to increase the underlying ring by a “gap” of at least 40 bits, to ensure 2^{-40} soundness error. Intuitively, we can construct covertly secure protocols by using a smaller ring gap s s.t. the soundness error is bounded by $1 - \epsilon$. We use $\epsilon = 50\%$ for a concrete example. Given an arithmetic circuit over $\mathbb{Z}_{2^{64}}$ of size $m = 2^{20}$, we can set $s = 3$ by using MACs [1, Lemma 2]. Compared to the malicious setting where 40-bit statistical security is required (where $s = 46$), the communication cost is reduced from 220 bits to 134 bits per gate. If we use DZKPs, we can set $s = 12$ [49, Lemma G.3] (with the recursion parameter $q = 16$). The amortized communication cost is only 64 bits. We denote the above two approaches by $\text{Cvt-MAC}^{\text{E2e}}$ and $\text{Cvt-DZKP}_A^{\text{Wan}}$, respectively. $\text{Cvt-MAC}^{\text{E2e}}$ has a better end-to-end performance in LAN due to its relatively low computational cost, while $\text{Cvt-DZKP}_A^{\text{Wan}}$ can outperform $\text{Cvt-MAC}^{\text{E2e}}$ in WAN due to its low communication burden².

Interestingly, even a small relaxed soundness guarantee may significantly lower the demand on the size of ring gap s : even if we set ϵ to 95% in the above example, it suffices to set $s = 8$ by using MACs and $s = 12$ (with a larger recursion factor $q = 32$) by using DZKPs. Meanwhile, we notice another key fact: multiplications over 2^{64+s} (where $1 \leq s \leq 64$) is approximately $2 \sim 3\times$ slower than that over 2^{64} for modern CPUs, and a small or moderate ring size is sufficient for many real-world applications. For example, previous works [58], [44], [39] have adopted a ring size of approximately $32 \sim 41$ bits. In such cases, to achieve malicious security, the ring has to be expanded to 80 bits using MACs, and approximately 100 bits using DZKPs. However, if we turn to the covert model, we can safely use 8 or 12 extra bits even with $\epsilon = 95\%$, and the final expanded ring is still bounded in 64 bits. This directly leads to a $2 \sim 3\times$ reduction in computational burden.

²The soundness analyses and security proofs of $\text{Cvt-MAC}^{\text{E2e}}$ and $\text{Cvt-DZKP}_A^{\text{Wan}}$ naturally follow [1], [49], and thus we omit the details here.

Fig. 2: Workflow of Cvt-MAC^{On}.

B. A New Online-Efficient Approach

We notice that both the MAC-based and DZKP-based approaches are input-dependent protocols, i.e., the verification is conducted entirely after the inputs are known, and thus all the additional overheads are incurred online. This may be suboptimal for settings where parties can preprocess and demand maximal online efficiency. Another previous work ASTRA [19] adopts the relatively online-efficient triple-based approach to achieve malicious security; however, the online cost is still two times that of the semi-honest protocol, and the offline communication burden is heavy. This also holds for another SOTA triple-based work [23]. We thus aim to design an optimized protocol with a lightweight online cost and a moderate offline burden.

We start from the preprocessing model of ASTRA [19]. Our key observation is that the semi-honest protocol of ASTRA [19] is asymmetric: unlike the RSS-based protocol [3] where the parties act all the same, in ASTRA P_0 distributes offline randomness for P_1, P_2 to consume online. Motivated by this, we design different verification mechanisms for P_0 and P_1, P_2 with an asymmetric use of MACs, and achieve a lightweight online cost, with even lower total communication burden than the normal MAC-based protocol [1]. We denote this approach by Cvt-MAC^{On}, and elaborate it below.

Verification on P_0 . We now take a deep look at P_0 's behavior in the semi-honest protocol of ASTRA. For each multiplication gate computing $[z] = [x] \cdot [y]$, P_0 additively shares $r_{xy} = r_x \cdot r_y$ to P_1, P_2 . As P_1, P_2 hold $[r_x], [r_y]$, we can view $([r_x], [r_y], [r_{xy}])$ as an additively shared multiplication triple. Since P_0 is silent during the online circuit evaluation phase, and the output reconstruction procedure is robust, the only way it can corrupt the final outputs is to offer incorrect multiplication triples. Thus, it suffices for P_1, P_2 to check the correctness of these triples to check P_0 's behavior. This can be handled by the $\mathcal{F}_{\text{VrfyIP}}^\epsilon$ functionality (see Section III-D).

Specifically, as P_1, P_2 hold additive shares of these triples which are known by P_0 , they can locally covert them into replicated secret shared shares (by locally setting an extra additive share to zero), and then invoke $\mathcal{F}_{\text{VrfyIP}}^\epsilon$ to check these triples.

We instantiate $\mathcal{F}_{\text{VrfyIP}}^\epsilon$ using DZKPs by adapting the involved parameters as in Section IV-A (a formal description is given in Appendix D). Due to the sublinear communication cost inherited from DZKPs, the verification procedure incurs no (amortized) communication burden.

Verification on P_1, P_2 . Recall that to authenticate a computation over $\mathbb{Z}_{2^k}$ using MACs, the ring has to be lifted to $\mathbb{Z}_{2^{k+s}}$ (but only the least significant k bits of the outputs are effective). In the normal usage of MACs, the parties together sample a random shared MAC key $[\alpha]$ where $\alpha \in \mathbb{Z}_{2^{k+s}}$; then each input v is shared over $\mathbb{Z}_{2^{k+s}}$ and authenticated by computing $[\tilde{v}] :=_{k+s} [\alpha] \cdot [v]$. For each multiplication gate computing $[z] :=_{k+s} [x] \cdot [y]$, the parties authenticate z by computing $[\tilde{z}] :=_{k+s} [\tilde{x}] \cdot [y]$ (or $[\tilde{z}] :=_{k+s} [x] \cdot [\tilde{y}]$, equivalently). After evaluating all multiplication gates, they sample random secret coefficients and perform a random linear combination on all $\{[\tilde{v}], [\tilde{z}]\}$ and also the authenticated $\{[\tilde{v}], [\tilde{z}]\}$ (by securely computing two inner products), then opening the MAC key α to check the consistency of the two resulting shares.

Our key insight here is that, as we now only concern the correct execution of P_1, P_2 , we can view P_0 as a trusted party and let it sample the MAC key α and additively share necessary correlated randomness to P_1, P_2 , who then are able to compute an authenticated $\tilde{z}$ in the ASS form rather than the RSS form. This allows P_1, P_2 to obtain $[\tilde{z}]$ in a non-interactive way, and thus incurs no extra (amortized) online communication cost over the semi-honest protocol. We detail it below.

(i) Asymmetric Usage of MACs: Observe that for each multiplication gate computing $[z] := [x] \cdot [y]$, we have $\alpha z = \alpha(\delta_x + r_x)(\delta_y + r_y) = \alpha\delta_x\delta_y + \alpha\delta_xr_y + \alpha\delta_yr_x + \alpha r_{xy}$. Let $\tilde{z} := \alpha z$ be the authenticated value of z . By defining $\tilde{r}_x := \alpha r_x$, $\tilde{r}_y := \alpha r_y$, and $\tilde{r}_{xy} := \alpha r_{xy}$, we have

$$[\tilde{z}] = \delta_x\delta_y[\alpha] + \delta_x[\tilde{r}_y] + \delta_y[\tilde{r}_x] + [\tilde{r}_{xy}]. \quad (1)$$

Therefore, for P_1, P_2 to obtain $[\tilde{z}]$, it suffices for P_0 to distribute the authenticated randomness $[\tilde{r}_x], [\tilde{r}_y], [\tilde{r}_{xy}]$ to P_1, P_2 , which can be done in the preprocessing phase.

However, the above process still requires an extra communication of $3m$ (where m is the circuit size) ring elements over the semi-honest protocol, the same as the normal usage of MACs. Below we show how to further reduce it to $2m + 1$.

(ii) Compressing Communication: Let $\{z^{(i)}\}_{i=1}^m$ be the outputs of m multiplication gates produced in the online phase. Suppose there are m random public coefficients $\{\eta^{(i)}\}_{i=1}^m$ over $\mathbb{Z}_{2^{k+s}}$. We let the parties apply a random linear combination to $\{[\tilde{z}^{(i)}]\}_{i=1}^m$ and obtain a combined result $[\tilde{w}] := \sum_{i=1}^m \eta^{(i)} \cdot [\tilde{z}^{(i)}]$. Now we define $\tilde{w} := \sum_{i=1}^m \eta^{(i)} \cdot \tilde{z}^{(i)}$, which is an authenticated value of w . Naively, P_1, P_2 are able to locally obtain $[\tilde{w}]$ given the authenticated randomness

$[\tilde{r}_x^{(i)}], [\tilde{r}_y^{(i)}], [\tilde{r}_{xy}^{(i)}]$ and the public coefficients. However, we notice that the communication of all $[\tilde{r}_{xy}^{(i)}]$ can be saved by sharing only one combined value. Specifically, by Equation (1), we have

$$[\tilde{w}] = \sum_{i=1}^m \eta^{(i)} (\delta_x^{(i)} \delta_y^{(i)} [\alpha] + \delta_x^{(i)} [\tilde{r}_y^{(i)}] + \delta_y^{(i)} [\tilde{r}_x^{(i)}]) + \sum_{i=1}^m \eta^{(i)} \cdot [\tilde{r}_{xy}^{(i)}].$$

Let $\tilde{w}_0 := \sum_{i=1}^m \eta^{(i)} \tilde{r}_{xy}^{(i)}$. Clearly P_0 can compute and distribute $\tilde{w}_0$ in the preprocessing phase. Then P_1, P_2 can locally derive $[\tilde{w}]$ given $\{[\tilde{r}_x^{(i)}]\}_{i=1}^m, \{[\tilde{r}_y^{(i)}]\}_{i=1}^m$ and $[\tilde{w}_0]$, which indicates P_0 only needs to share $2m + 1$ elements.

The public random coefficients should be known by P_1, P_2 after all multiplication gates are computed. The parties can call $\mathcal{F}_{\text{coin}}$ to generate them; however, we notice that it can be adapted to the preprocessing model by P_0 first sharing a random seed sd to P_1, P_2 together with a commitment of sd , and derive all $\{\eta^{(i)}\}_{i=1}^m$ using PRG with sd in the preprocessing phase, and then P_1, P_2 can recover sd to derive $\{\eta^{(i)}\}_{i=1}^m$ after computing all multiplication gates in the online phase. In this way, P_0 can still keep silent during the online phase until the reconstruction procedure.

(iii) Consistency Check: The remaining task is to check consistency between $[w]$ and $[\tilde{w}]$. To this end, we let P_1, P_2 derive another additive sharing of $\tilde{w}' := \alpha \cdot w$ from $[\alpha]$ and $[w]$ with the help of P_0 . Given that $[\tilde{w}'] = [\alpha] (\delta_w + [r_w]) = \delta_w [\alpha] + [\alpha \cdot r_w]$ and $r_w = \sum_{i=1}^m \eta^{(i)} r_z^{(i)}$, we ask P_0 to compute r_w and then share $[\tilde{r}_w] := [\alpha \cdot r_w]$ in the preprocessing phase; then P_1, P_2 are able to locally compute $[\tilde{w}']$ in the online phase. After this, the final consistency check can be performed by calling $\mathcal{F}_{\text{checkzero}}$ on $[o] := [\tilde{w}] - [\tilde{w}']$.

(iv) Additional Check on P_0 : Note that a corrupted P_0 may share incorrect $\{\tilde{r}_x^{(i)}\}_{i=1}^m, \{\tilde{r}_y^{(i)}\}_{i=1}^m, \tilde{w}_0$ or $\tilde{r}_w$ to P_1, P_2 , and try to extract extra information according to the behaviors of P_1, P_2 (i.e., whether they abort or not). Thus, it's necessary for P_1, P_2 to additionally check the authenticated randomness. Similarly, we can invoke $\mathcal{F}_{\text{rfyIP}}^\epsilon$ on these triples. However, to avoid heavy computational burdens in the preprocessing phase, we opt for checking these values online in a more lightweight way: after all the multiplication gates are checked, we let P_1, P_2 directly open α to check the consistency of these values. The communication cost can be saved by applying a random linear combination and then invoking $\mathcal{F}_{\text{checkzero}}$ on the combined result. We claim that opening α doesn't violate privacy even if P_1 or P_2 is corrupted, as the procedure is performed after authenticating P_1 and P_2 's computations. If a corrupted P_1 or P_2 can successfully cheat in the previous steps (by introducing some additive error), it must have already guessed necessary information about α . Thus the revealing of α doesn't give it any extra information.

We present the workflow of our protocol in Figure 2, and give a formal description as well as a security proof in Appendix E. In summary, our protocol only requires communicating about $3m$ in the offline phase and $2m$ ring elements in the online phase using PRG (as a contrast, the original MAC-based protocol [1] incurs a total communication of $6m$ ring elements in the online phase).

V. CHEATING DETECTION ON BINARY CIRCUITS

Unlike arithmetic circuits, using MACs over a larger ring to authenticate AND gate computation would significantly degrade the online efficiency (as ring operations are more expensive than Boolean ones). Therefore, we opt for the SOTA triple-based approach [8], and design a new cut-and-choose paradigm tailored for the covert model with better online/end-to-end efficiency. We also adapt the DZKP-based approach [9], [12] to the covert model, considering its optimal sublinear communication cost.

A. Adapting DZKPs with Smaller Field Size

As DZKPs cannot directly apply to $\mathbb{Z}_2$ due to their large soundness error, previous maliciously secure protocols [9], [12] proposed to lift the verification onto a sufficiently large binary extension field $GF(2^n)$ to guarantee negligible soundness error. By using the recursive techniques, the soundness error is bounded by $O(\frac{\log m}{|\mathbb{F}|})$ [12, Theorem 7.1]. In our case, we only need the extension field $GF(2^n)$ to be moderately large such that the soundness error is bounded by $1 - \epsilon$. Concretely, setting $n = 4, 8$ or 16 is sufficient for most choices of ϵ . We denote the adapted protocol by $\text{Cvt-DZKP}_B^{\text{wan}}$.

Note that the recent alternative DZKP protocol [48] over Mersenne prime fields (recall Section II-A) cannot apply, as it has a restriction on circuit size to avoid overflow, and a small field size here would significantly limit practical applications.

B. Covertly Secure Cut-and-Choose Method

We now introduce our triple-based approaches with a new cut-and-choose paradigm tailored for the covert model.

Achieving Efficient Online with Lower Offline Costs. We first review how to compute an AND gate using a random triple. Assume the parties are given a random AND triple $([a], [b], [c])$ where $c = a \cdot b$; to compute an AND gate $[z] := [x] \cdot [y]$, the parties locally compute $[\rho] := [x] \oplus [a]$ and $[\sigma] := [y] \oplus [b]$, then opening ρ, σ and computing $[z]$ by

$$[z] = \rho \cdot \sigma \oplus \rho \cdot [b] \oplus \sigma \cdot [a] \oplus [c]. \quad (2)$$

Note that the online computation is (relatively) lightweight given prepared random triples, and thus we use this approach to construct our online-efficient protocol $\text{Cvt-Triple}^{\text{On}}$. Since the offline cost is heavy, we aim to optimize the offline procedure with lower costs.

First recall that the previous work [32] proposed to use the classical cut-and-choose paradigm to generate offline random triples. Assume there are N AND gates in the Boolean circuit. The parties first run the RSS-based semi-honest AND protocol to generate $NB + C$ initial random triples where B, C are parameters. Then they randomly shuffle these triples, open and check the first C of them, and then divide the unopened triples into N buckets, each of size B . For each bucket, the parties perform the bucket-check procedure, where the first triple is checked using the remaining $B - 1$ triples respectively.³ If all

³To check the first triple (say $([x], [y], [z])$) with another one (say, $([a], [b], [c])$), the parties compute the RHS of Equation (2), and check if the difference of the result and $[z]$ is a sharing of zero.

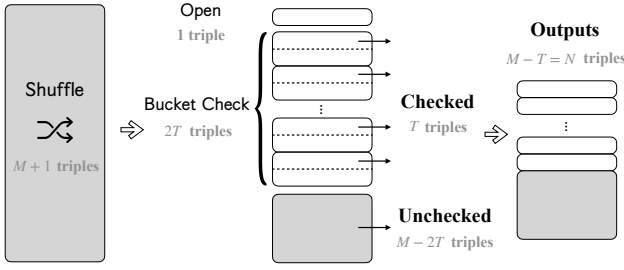


Fig. 3: Triple generation procedure of $\text{Cvt-Triple}^{\text{On}}$. (M, T are parameters s.t. $M - T = N$.)

the checks are passed, they output all the first triples in these buckets, yielding N validated triples in total. As analyzed in [32], the soundness error is bounded by $2^{-\sigma}$ provided that $\sigma \leq \frac{(BN+B)!}{N \cdot B! \cdot (BN)!}$ and $C = B$. For example, for $N = 2^{20}$, we can set $B = C = 3$ to achieve $\sigma = 40$ bit security, and this requires a (global) communication of 21 bits per gate.

In the covert model, a naive method is to use the smallest bucket size $B = 2$, and this would reduce the offline communication cost to 12 bits per gate, with a soundness error of roughly $\frac{1}{N}$. However, we notice that the above soundness guarantee may exceed most of practical requirements on ϵ , even for a moderate N . For example, the soundness guarantee will be at least 99% for $N > 100$; for one expecting a deterrence factor of 50%, the exceeded soundness guarantee (and the corresponding overheads) would be unnecessary. To minimize the costs in such cases, we optimize the above protocol by only performing the bucket-check procedure on a random part of the triples. To be more specific, we first choose proper parameters M, T s.t. $M - T = N$ and $0 < T \leq \frac{M}{2}$. Then we let the parties generate $M + 1$ random semi-honest triples, and then randomly shuffle them. After this, the parties open and check the first one triple. Then they divide the first $2T$ out of the unopened M triples into T buckets each of size 2, and perform the bucket-check procedure on those T buckets. If all the checks are passed, the parties output the checked T triples in the buckets and also all the remaining unchecked $M - 2T$ triples, generating $M - T = N$ triples in total.

We give an illustration of the above procedure in Figure 3, and present a formal description and soundness analysis in Appendix F. We can choose M, T according to the deterrence factor ϵ . Concretely, for $\epsilon = 50\%$, we can set $M = \frac{4(N+1)}{3}$ and $T = \frac{M}{4} + 1$. Compared to the malicious setting, the offline communication cost is reduced from 21 bits to 6 bits. For $\epsilon = 75\%$ and $\epsilon = 87.5\%$, we can set $M = \frac{16N}{9}$ and $\frac{8N}{5}$, with $T = \frac{7M}{16}$ and $\frac{3M}{8}$, and the offline communication cost is roughly 8 and 10 bits, respectively.

Achieving Higher End-to-End Efficiency. The previous work [2] has proposed a variant of the cut-and-choose approach that has better end-to-end efficiency, by using a smaller bucket size in the preprocessing phase at the cost of a slightly higher online cost. Specifically, the parties only generate $N(B - 1) + C$ random triples, and perform the bucket-check procedure as in [32]. Unlike [32], now the output triples are not guaranteed to be validated (the soundness error is not

negligible, as B is now reduced by 1). Then in the online phase, the parties first compute N AND gates using the semi-honest multiplication protocol, then apply a random shuffle to these online triples, which are then checked with those N random offline triples. Intuitively, the random shuffle applied to the online triples guarantees that the overall soundness guarantee is nearly the same as [32].

Following [2], we design a variant of the cut-and-choose protocol in the covert model with higher end-to-end efficiency, denoted by $\text{Cvt-Triple}^{\text{E2e}}$. Similarly, we reduce the number of the initial triples by N : instead of $M + 1$, we now let the parties generate $M + 1 - N = T + 1$ (as $M - T = N$) initial triples and randomly shuffle them. Then the parties open and check the first triple as above; the remaining T triples will be used for later online verification. After computing N online triples, the parties apply a random shuffle to them, and divide these triples into two parts: the first T online triples will be paired and checked with those T random offline triples, and the remaining $N - T = M - 2T$ online triples will not be checked. In this way, the overall soundness guarantee is still guaranteed to be at least ϵ with proper parameters, and the number of initial triples to be generated is directly reduced by N , leading to higher end-to-end efficiency. The soundness analysis of $\text{Cvt-Triple}^{\text{E2e}}$ naturally follows that of $\text{Cvt-Triple}^{\text{On}}$.

Remark 5.1: Our new cut-and-choose paradigm tailored for the covert model can be extended to more types of correlated randomness beyond AND triples, such as (e)daBits [30], truncation pairs [54], dot-product/matrix triples, and so on. This enables us to construct concretely efficient covert protocols for mixed circuits.

VI. CHEATER IDENTIFICATION

In this section, we show how to identify cheaters with public verifiability. Differently from previous works [62], [31], we observe that in the three-party honest-majority setting, we can identify cheaters without introducing any other cryptographic primitives (other than signatures and commitments) for some protocols with specific properties. For the other protocols, we can identify cheaters by using CP-NIZK proofs.

A. Lightweight Cheater Identification

Our key observation is, in the three-party honest-majority setting, if the parties can identify an honest party after detecting cheating behavior, then the honest party can act as a trusted judge and find out the cheater by checking the other parties' views. This requires the two parties to reveal their views to the honest party (which doesn't violate privacy since the party is honest); to guarantee the two parties' views are not forged, we require the parties to first commit to their input and random tape, and sign every sent message during the execution of the protocol. The binding property of the commitment scheme and the unforgeable property of the signature scheme ensure that any forged view will lead to a failure in verifying the commitments or signatures. Later we'll show how to identify an honest party for specific protocols.

Notice that the abortion can also occur in the final reconstruction procedure, where a malicious party sends an

incorrect share during reconstruction. To identify the cheater in such cases, we use techniques from [12] to let the parties authenticate each output with their individual secret MAC key, and the parties will securely compute the original outputs of the circuit as well as the authenticated outputs w.r.t. their MAC keys. The insight here is that, if a cheating party P_i sends an incorrect share to P_j during reconstruction, then it must successfully guess correct information about P_j 's secret MAC key to pass its MAC check, otherwise it will be directly identified as the cheater. As the authentication is only applied to the final outputs, the extra overhead is nearly negligible.

(i) Identifying an Honest Party for DZKP-based Approach. We follow [12] to identify an honest party for DZKP-based approaches. If an abortion happens during the verification of P_j 's behaviors, there are two possible cases: (1) the prover P_j is indeed malicious, (2) one of the two verifiers is cheating. To determine an honest party, we first identify a conflict pair, i.e., a pair of two parties where one of them must be malicious. To this end, we let the prover P_j accuse a verifier P_t where $t \in \{j-1, j+1\}$, and then P_j, P_t will be viewed as a conflict pair. It's straightforward to see that in the first case the malicious prover must be included in the conflict pair. As for the second case, the key property we utilize is "re-computable verification": since an honest prover knows all the distributed witnesses held by each verifier, it can re-compute all the messages that should be sent by the two verifiers, and thus is able to tell exactly which verifier is cheating. Therefore, the conflict pair still includes the cheating party. As we are in the honest-majority setting, the left one verifier that is not accused must be honest.

(ii) Identifying an Honest Party for Asymmetric MAC-based Approach. For the asymmetric MAC-based approach, we make use of the asymmetric preprocessing property to identify an honest party: if P_0 's behavior has been successfully verified, then P_0 can be identified as an honest party, otherwise we let P_0 accuse another party, and the left one party will be viewed as honest. Note that if a malicious P_1 or P_2 tries to cheat during the verification of P_0 , it can only manipulate two procedures: (1) the offline DZKP verification on P_0 's randomness $\{([r_x^{(i)}], [r_y^{(i)}], [r_{xy}^{(i)}])\}_{i=1}^m$, or (2) the online consistency check of the authenticated randomness $\{[\tilde{r}_x^{(i)}]\}_{i=1}^m, \{[\tilde{r}_y^{(i)}]\}_{i=1}^m, [\tilde{w}_0]$ and $[\tilde{r}_w]$. Under any circumstance, an honest P_0 can always tell the cheater since it knows all the involved randomness.

B. Achieving Public Verifiability

We observe that in the three-party honest-majority setting, public verifiability follows naturally after identifying the cheater. Suppose P_j is identified to be malicious. As the other two parties must be honest, we can let each honest party sign an accusation message (corrupted, i) message and exchange it with each other, and the Blame algorithm just outputs the two signed accusation messages as a certificate. After this, any third party running the Judge algorithm just validates the two signatures w.r.t. the accusation message; as the signatures are generated by the honest parties, the check must pass, and the

Judge algorithm will output the identity of the cheating party correctly. In Appendix G-B we give formal descriptions and security proofs of the two algorithms.

VII. ECHO: COVERTLY SECURE PPML FRAMEWORK

In this section, we present a covertly secure PPML framework with public verifiability. To this end, we adapt existing protocols for PPML primitives to covert security.

A. Fixed-Point Representation

We use fixed-point arithmetic, where a k -bit signed fixed-point $\tilde{x}$ is represented with two parts: an e -bit integer part and a d -bit decimal part s.t. $k = e + d$. We map the fixed-point number $\tilde{x}$ into an element x in $\mathbb{Z}_{2^k}$ by setting $x := 2^d \cdot \tilde{x}$, and addition and multiplication operations are mapped accordingly.

B. Share Conversion

In the PPML framework, we sometimes need to transform the shares back and forth over $\mathbb{Z}_2$ and $\mathbb{Z}_{2^k}$, referred to as A2B and B2A, respectively. We consider a typical case of B2A, where a shared bit is converted into the arithmetic domain, commonly referred to as Bit2A.

We follow the previous work [54] to transform arithmetic shares into binary shares (A2B). Given an arithmetic sharing $\llbracket x \rrbracket^A = (x_0, x_1, x_2)$, to compute a binary secret sharing of x , we let each party P_j decompose x_j into bits $x_j^{(0)}, \dots, x_j^{(k-1)}$, and then the parties locally generate the binary sharing of these bits, denoted by $\llbracket \vec{x}_j \rrbracket^B := (\llbracket x_j^{(0)} \rrbracket^B, \dots, \llbracket x_j^{(k-1)} \rrbracket^B)$. Then they run binary adders to compute $x_0 + x_1 + x_2$, taking input as binary sharings $\llbracket \vec{x}_0 \rrbracket^B, \llbracket \vec{x}_1 \rrbracket^B$ and $\llbracket \vec{x}_2 \rrbracket^B$. By carefully designing the binary adders (i.e., using fuller adders and parallel prefix adders), the previous work [54] can reduce three addition operands into two in one round, and obtain the final binary sharing in $O(\log k)$ rounds, with $O(k)$ communication cost. In the covert model, we can apply our covertly secure protocol for binary circuits to obtain a covertly secure A2B protocol.

For Bit2A, we utilize the fact that for any $x, y \in \mathbb{Z}_2$, we have $x \oplus y = x + y - 2xy$. Given a shared bit $\llbracket b \rrbracket^B = (b_0, b_1, b_2)$, we have $b = b_0 \oplus b_1 \oplus b_2$. Thus, to obtain an arithmetic sharing of b , we let the parties first locally generate replicated secret sharings $\llbracket b_0 \rrbracket^A, \llbracket b_1 \rrbracket^A, \llbracket b_2 \rrbracket^A$ over $\mathbb{Z}_{2^k}$, and then invoke the secure arithmetic multiplication protocol to compute each XOR operation. This requires two secure multiplications, and we can directly apply our covertly secure protocol here.

C. Truncation

Truncation is typically invoked after arithmetic multiplications to adjust the decimal part of the results. It takes as input a sharing $\llbracket x \rrbracket^A$ of integer $x \in \mathbb{Z}_{2^k}$, and outputs a sharing of x' where $x' = \lfloor x/2^d \rfloor$, and d is the bit length of the decimal part in the fixed-point representation. In this framework, we adopt probabilistic truncation, where it rounds $x/2^d$ to $x' = \lfloor x/2^d \rfloor + w$ where $w \in \{0, 1\}$ is biased towards the nearest integer to $x/2^d$.

In this work, we opt for probabilistic truncation based on the online-efficient approach from prior works [25], [43]. We

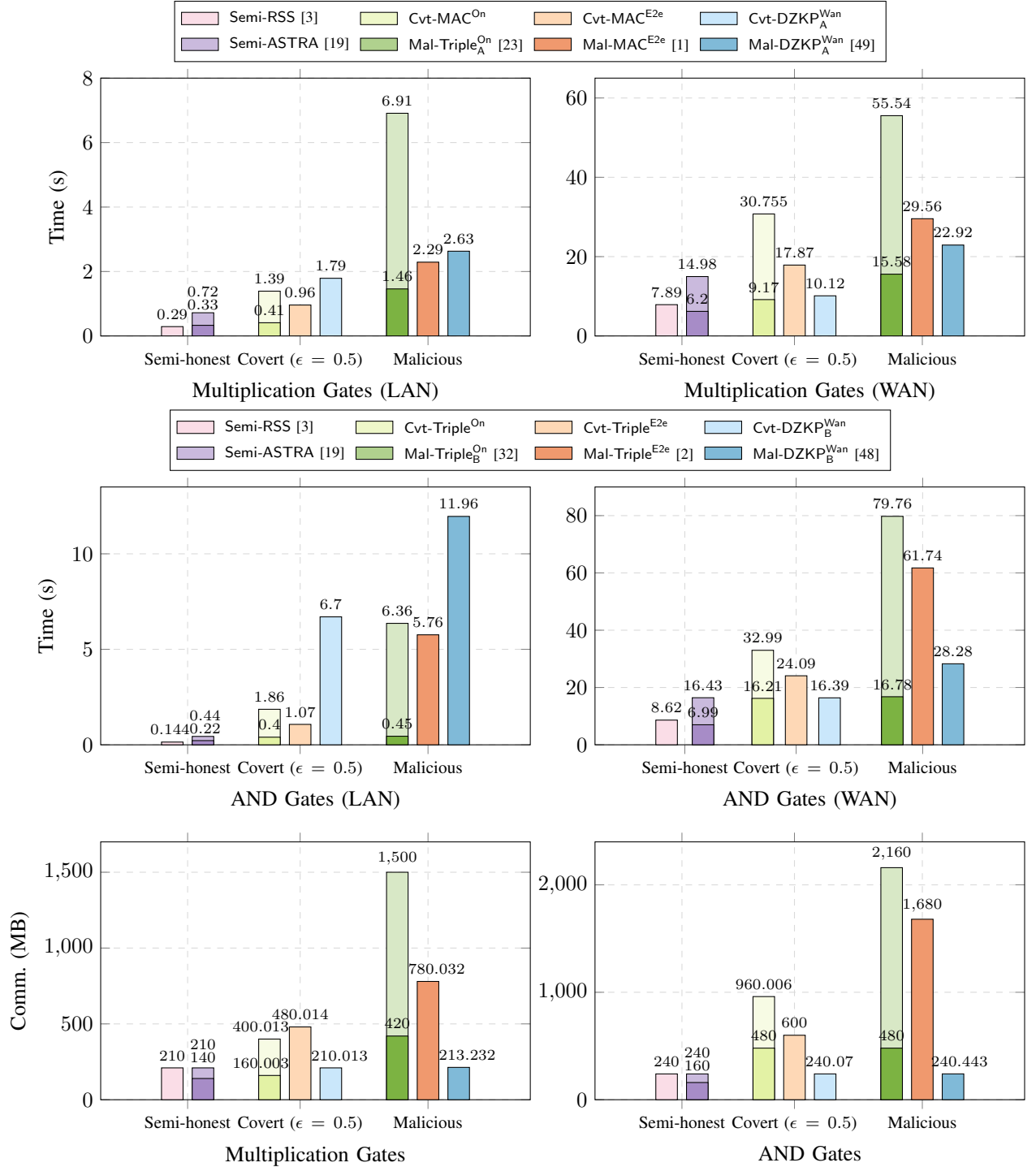


Fig. 4: Time (s) and communication cost (MB) for computing depth-10 circuits where each layer consists of 2^{20} Multiplication Gates over $\mathbb{Z}_{2^{52}}$ or 2^{20} 64-bit parallel AND gates. For protocols in the preprocessing model, the darker (bottom) portion of each bar represents the online cost, while the total bar represents the end-to-end cost. The legends for the communication cost plots are reused from the corresponding time cost plots.

assume a “small-gap” on the truncated value x to guarantee the correctness of the truncation results [25], [43], [26], [46], i.e., by requiring that the most significant bit of x is zero. (this can be done by adding x with 2^{k-1} before truncation then subtracting 2^{k-d-1} from the truncated value afterward [24], [43]). This approach involves the parties preparing random shared bits $\llbracket r^{(i)} \rrbracket$ and $r = \sum_{i=1}^k \llbracket r^{(i)} \rrbracket$ over $\mathbb{Z}_{2^k}$ in the preprocessing phase. Following [25], we let two parties propose random bits and XOR them using our covert security protocol, and additionally check each secret is from $\mathbb{Z}_2$ using the verification techniques from [26] (more details refer to [24], [26]). To address the security concerns raised by Li et al. [48], we adopt the approach from [61], with a slight modification to the functionality to ensure security.

D. Comparison

The comparison operation is commonly used in activation functions (like ReLU) and Max Pooling layers in neural networks. A secure comparison protocol over ring $\mathbb{Z}_{2^k}$ takes as input two sharings $\llbracket a \rrbracket$, $\llbracket b \rrbracket$, and outputs the sharing of z where $z = 0$ denotes $a \geq b$ and $z = 1$ otherwise. We follow previous works [54] to decompose secure comparison into the following steps: each party locally computes $\llbracket z \rrbracket^A = \llbracket a \rrbracket^A - \llbracket b \rrbracket^A$; then the value $\llbracket z \rrbracket^A$ is converted to a binary share $\llbracket z \rrbracket^B$ via the A2B operation, and the most significant bit $\llbracket z_0 \rrbracket^B$ is extracted and converted back to arithmetic sharing using Bit2A, yielding $\llbracket z_0 \rrbracket^A$ as the output of the comparison.

E. (E)daBits

A daBit (doubly-authenticated Bit) [60] is a random bit shared over both the arithmetic and the binary domain, i.e., $(\llbracket r \rrbracket^A, \llbracket r \rrbracket^B)$ where $r \in \{0, 1\}$. An EdaBit (Extended daBit) [30] is a generalization of daBit, consisting of n random bits shared over the binary domain $\llbracket r^{(0)} \rrbracket^B, \dots, \llbracket r^{(n-1)} \rrbracket^B$ and an integer r shared over the arithmetic domain s.t. $r = \sum_{i \in [n]} r^{(i)} \cdot 2^i$. (E)daBits are commonly used in PPML to help evaluate non-linear functions, such as truncation, comparison, share conversion, etc. The generation of (E)daBits typically involves the cut-and-choose paradigm, and in the covert model we can accordingly apply our cut-and-choose paradigm (described in section V-B) to the generation procedures.

VIII. EVALUATION

A. Experimental Setup

We implemented our covertly secure protocols based on the MP-SPDZ framework [43].⁴ Our experiments were run on three different cloud instances under a LAN, each equipped with a 32-core Intel(R) Xeon(R) Platinum 6982P-C 3GHz CPU and 256GB memory. We report results in the LAN and an emulated WAN with 100Mbps bandwidth and 40ms one-way latency using the Linux command *tc*. All the microbenchmarks were run in a single thread, and all PPML tasks were run in 16 threads.

To comprehensively evaluate our covertly secure protocols, we use the two semi-honest protocols based on RSS [3]

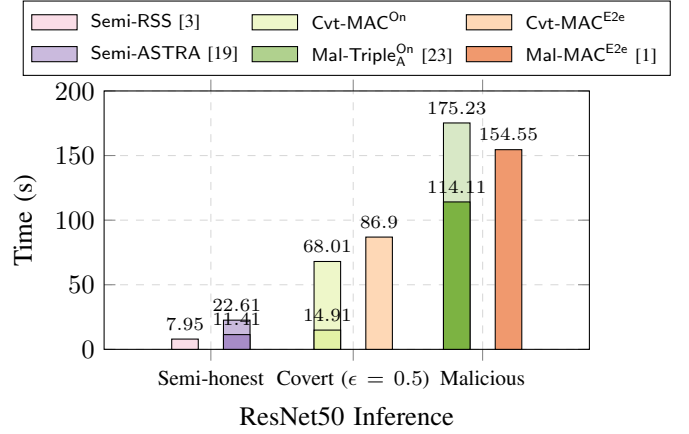


Fig. 5: LAN PPML (ResNet50) runtime. For protocols with offline/online phases (ASTRA, Cvt^{On}, Mal-MAC^{E2e}+Mal-Triple^{On}), the darker portion denotes online cost; the full bar indicates end-to-end cost.

and ASTRA [19] sharing schemes (denoted by Semi-RSS, Semi-ASTRA respectively) as baselines, and compare with SOTA maliciously secure protocols. The deterrence factor in our covert security protocol is set to 50%.

B. Microbenchmark Results

We compare with maliciously secure MAC-based [1], DZKP-based [49] and triple-based [23] protocols for arithmetic circuits (denoted by Mal-MAC^{E2e}, Mal-DZKP^{Wan}, Mal-Triple^{On} respectively). For Boolean circuits, we compare with maliciously secure triple-based [32], [2] and DZKP-based protocols [48] (denoted by Mal-Triple^{On}, Mal-Triple^{E2e}, Mal-DZKP^{Wan} respectively). We show the time cost and communication cost of the tested protocols in Figure 4, and give detailed analyses below.

Multiplication Gates. In the LAN setting, the online time of Cvt-MAC^{On} is only $1.42\times$ and $1.24\times$ of the semi-honest protocols Semi-RSS and Semi-ASTRA, respectively; compared to the SOTA online-efficient malicious protocol Mal-Triple^{On}, Cvt-MAC^{On} achieves a $3.56\times$ and $5.54\times$ speedup in the online and offline phase, respectively. Meanwhile, Cvt-MAC^{E2e} can achieve a $2.38\times$ speedup compared to Mal-MAC^{E2e} in end-to-end efficiency.

Under the WAN setting, as Cvt-MAC^{On}'s online communication remains lightweight, it incurs only $1.16\times$ online overhead over Semi-RSS and $1.04\times$ over Semi-ASTRA. For end-to-end performance, DZKP-based protocols outperform the others due to their low communication burden, and our covertly secure protocol Cvt-DZKP^{Wan} is $2.27\times$ faster than the maliciously secure counterpart Mal-DZKP^{Wan} due to the reduction of the size of the computation ring.

The communication cost of Cvt-MAC^{E2e} is only $0.62\times$ of Mal-MAC^{E2e} by using a smaller ring size. Moreover, the online communication cost of Cvt-MAC^{On} is just $0.76\times$ and $1.14\times$ that of Semi-RSS and Semi-ASTRA, respectively; compared to the maliciously secure protocol Mal-Triple^{On}, the online and offline communication cost of Cvt-MAC^{On} is only $0.38\times$ and $0.22\times$, respectively.

⁴Our code is in <https://anonymous.4open.science/r/Covert-3PC-5E45>.

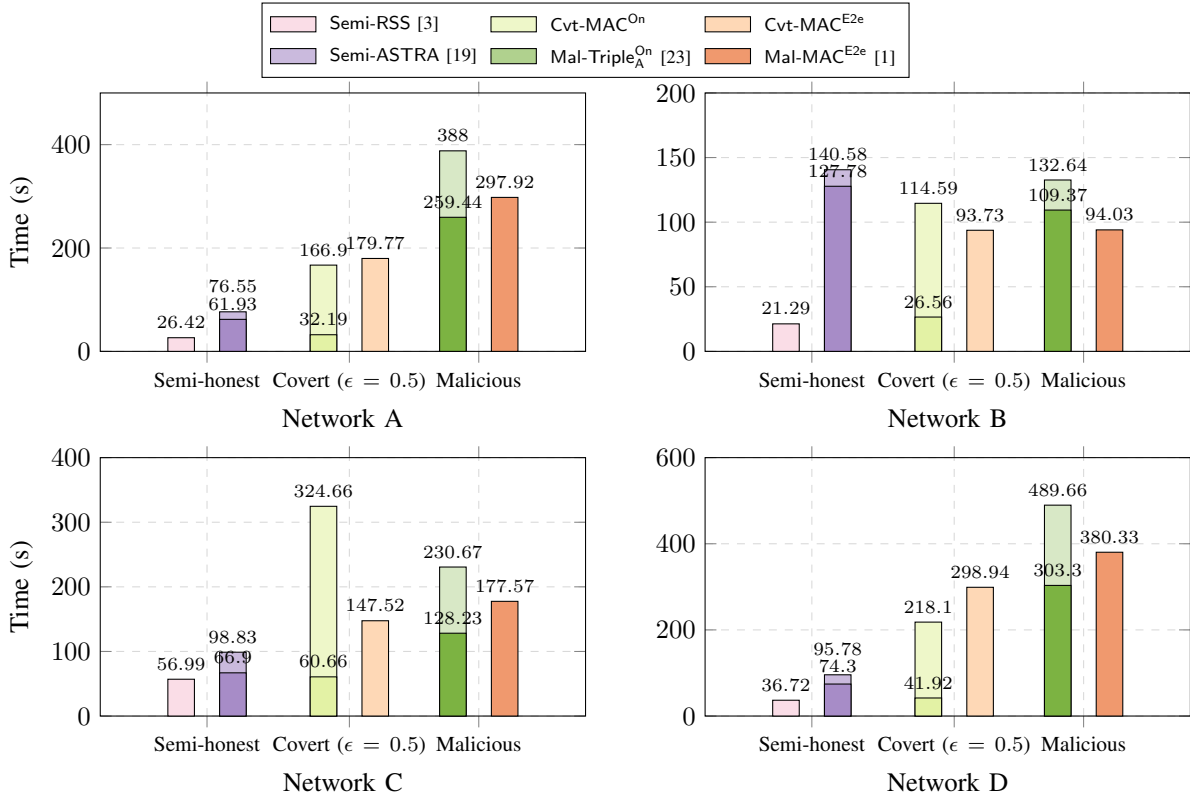


Fig. 6: LAN PPML runtime for training on different networks. For protocols with offline/online phases (ASTRA, Cvt-MAC^{On}, Mal-Triple^{On}), the darker portion denotes online cost; the full bar indicates end-to-end cost.

AND Gates. In the LAN setting, Cvt-Triple^{On} and the SOTA online-efficient malicious protocol Mal-Triple^{On} have similar online efficiencies (as their online procedures are the same); nevertheless, Cvt-Triple^{On} reduces the offline time by $2.89\times$ due to our tailored cut-and-choose paradigm. As for end-to-end performance, Cvt-Triple^{E2e} is $5.36\times$ faster than the maliciously secure counterpart [2] Mal-Triple^{E2e}.

In the WAN setting, the DZKP-based protocols show advantages: the overall running time of our covertly secure protocol Cvt-DZKP^{Wan} is only $1.9\times$ of Semi-RSS, and is even slightly faster than Semi-ASTRA; compared with the maliciously secure counterpart Mal-DZKP^{Wan}, Cvt-DZKP^{Wan} is $1.73\times$ faster.

On communication, both Cvt-Triple^{On} and Cvt-Triple^{E2e} benefit from our new cut-and-choose paradigm: Cvt-Triple^{On} reduces the communication cost of Mal-Triple^{On} by $4.2\times$ in the offline phase, and Cvt-Triple^{E2e} reduces the overall communication cost of Mal-Triple^{E2e} by $2.8\times$.

C. PPML Task Results

We compare with semi-honest protocols ([3], [19]) and malicious protocols ([1] for arithmetic and [32], [2] for Binary). We evaluate ResNet50 inference in Figure 5 and training on Network A, B, C, and D in Figure 6.

ResNet50 Inference.

As shown in Figure 5, we evaluate ResNet50 inference on the ImageNet dataset under the LAN setting. Our online-efficient protocol Cvt-MAC^{On} achieves less than $1\times$ overhead compared to the semi-honest baselines. Compared to

the maliciously secure protocol Mal-Triple^{On}, our Cvt-MAC^{On} achieves a substantial $7.65\times$ speedup (114.11s vs. 14.91s in online time). For end-to-end efficiency, our protocol Cvt-MAC^{E2e} provides a $1.77\times$ speedup compared to the maliciously secure counterpart Mal-MAC^{E2e}.

NN Training.

As shown in Figure 6, we evaluate our protocols on four neural network training tasks (Networks A, B, C, and D) on MNIST dataset. We trained one epoch with batch size 128. Specifically, for Network B and C, we used only 5000 training samples to reduce the training time.

Our online-efficient protocol Cvt-MAC^{On} achieves overhead ranging from 6% to 25% compared to the semi-honest baseline Semi-RSS, demonstrating that covert security adds minimal cost over semi-honest protocols in practical PPML workloads. Compared to the maliciously secure protocol Mal-Triple^{On}, our Cvt-MAC^{On} is up to $8.06\times$ faster, showing significant performance advantages. For end-to-end efficiency, our Cvt-MAC^{E2e} achieves speedups of at most $1.66\times$ compared to the maliciously secure counterpart Mal-MAC^{E2e}.

We note that in some experiments, Semi-ASTRA exhibits slower performance than certain malicious baselines, which might be attributed to implementation-specific issues in MP-SPDZ rather than fundamental protocol limitations.

D. Cheater Identification

We use the classic CP-NIZK, LegoSNARK [18], to preliminarily test the cost of the general cheater identification

procedure. For an arithmetic circuit of 2^{20} multiplication gates, our experimental results show it only takes less than one millisecond to verify a proof, and the proof size is just 34 bytes. The prover time is relatively high, about tens of seconds. However, we emphasize that this cheater identification procedure is only conducted when using the symmetric MAC-based and triple-based approaches after a cheating attempt is detected. In other words, our general cheater identification protocol incurs zero overhead in the optimistic case.

IX. CONCLUSION

In this work, we thoroughly investigate the methods to achieve covert security for secure 3PC, and introduce the first covertly secure 3PC framework ECHO. ECHO includes six protocols for both arithmetic and Boolean circuits, tailored for end-to-end efficiency, online efficiency, and communication efficiency. We utilize commit-and-prove zero-knowledge proofs to achieve public verifiable cheater identification, which incurs no extra costs in the optimistic case. We implemented ECHO and proved that our protocol achieves covert security and is highly efficient, outperforming previous maliciously secure protocols.

REFERENCES

- [1] Mark Abspoel, Anders Dalskov, Daniel Escudero, and Ariel Nof. An efficient passive-to-active compiler for honest-majority mpc over rings. In *International Conference on Applied Cryptography and Network Security*, pages 122–152. Springer, 2021.
- [2] Toshinori Araki, Assi Barak, Jun Furukawa, Tamar Lichter, Yehuda Lindell, Ariel Nof, Kazuma Ohara, Adi Watzman, and Or Weinstein. Optimized honest-majority mpc for malicious adversaries — breaking the 1 billion-gate per second barrier. In *2017 IEEE Symposium on Security and Privacy (SP)*, pages 843–862, 2017.
- [3] Toshinori Araki, Jun Furukawa, Yehuda Lindell, Ariel Nof, and Kazuma Ohara. High-throughput semi-honest secure three-party computation with an honest majority. In Edgar R. Weippl, Stefan Katzenbeisser, Christopher Kruegel, Andrew C. Myers, and Shai Halevi, editors, *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security, Vienna, Austria, October 24–28, 2016*, pages 805–817. ACM, 2016.
- [4] Diego F. Aranha, Emil Madsen Bendsen, Matteo Campanelli, Chaya Ganesh, Claudio Orlandi, and Akira Takahashi. ECLIPSE: enhanced compiling method for pedersen-committed zkSNARK engines. In Goichiro Hanaoka, Junji Shikata, and Yohei Watanabe, editors, *Public-Key Cryptography - PKC 2022 - 25th IACR International Conference on Practice and Theory of Public-Key Cryptography, Virtual Event, March 8–11, 2022, Proceedings, Part I*, volume 13177 of *Lecture Notes in Computer Science*, pages 584–614. Springer, 2022.
- [5] Gilad Asharov and Claudio Orlandi. Calling out cheaters: Covert security with public verifiability. In Xiaoyun Wang and Kazuo Sako, editors, *Advances in Cryptology - ASIACRYPT 2012 - 18th International Conference on the Theory and Application of Cryptology and Information Security, Beijing, China, December 2–6, 2012, Proceedings*, volume 7658 of *Lecture Notes in Computer Science*, pages 681–698. Springer, 2012.
- [6] Thomas Attema, Ronald Cramer, and Lisa Kohl. A compressed $\mathbb{Z}$ -varisigma $\mathbb{Z}$ -protocol theory for lattices. In Tal Malkin and Chris Peikert, editors, *Advances in Cryptology - CRYPTO 2021 - 41st Annual International Cryptology Conference, CRYPTO 2021, Virtual Event, August 16–20, 2021, Proceedings, Part II*, volume 12826 of *Lecture Notes in Computer Science*, pages 549–579. Springer, 2021.
- [7] Yonatan Aumann and Yehuda Lindell. Security against covert adversaries: Efficient protocols for realistic adversaries. In *Theory of Cryptography, 4th Theory of Cryptography Conference, TCC 2007, Amsterdam, The Netherlands, February 21–24, 2007, Proceedings*, volume 4392 of *Lecture Notes in Computer Science*, pages 137–156. Springer, 2007.
- [8] Donald Beaver. Efficient multiparty protocols using circuit randomization. In *Annual International Cryptology Conference*, pages 420–432. Springer, 1991.
- [9] Dan Boneh, Elette Boyle, Henry Corrigan-Gibbs, Niv Gilboa, and Yuval Ishai. Zero-knowledge proofs on secret-shared data via fully linear pcps. In *Annual International Cryptology Conference*, pages 67–97. Springer, 2019.
- [10] Elette Boyle, Niv Gilboa, and Yuval Ishai. Function Secret Sharing. In Elisabeth Oswald and Marc Fischlin, editors, *Advances in Cryptology - EUROCRYPT 2015*, volume 9057, pages 337–367. Springer Berlin Heidelberg, Berlin, Heidelberg, 2015. Series Title: Lecture Notes in Computer Science.
- [11] Elette Boyle, Niv Gilboa, and Yuval Ishai. Function Secret Sharing: Improvements and Extensions. In *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*, pages 1292–1303, Vienna Austria, October 2016. ACM.
- [12] Elette Boyle, Niv Gilboa, Yuval Ishai, and Ariel Nof. Practical fully secure three-party computation via sublinear distributed zero-knowledge proofs. In *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security*, pages 869–886, 2019.
- [13] Elette Boyle, Niv Gilboa, Yuval Ishai, and Ariel Nof. Efficient fully secure computation via distributed zero-knowledge proofs. In *International Conference on the Theory and Application of Cryptology and Information Security*, pages 244–276. Springer, 2020.
- [14] Elette Boyle, Niv Gilboa, Yuval Ishai, and Ariel Nof. Sublinear gmw-style compiler for MPC with preprocessing. In Tal Malkin and Chris Peikert, editors, *Advances in Cryptology - CRYPTO 2021 - 41st Annual International Cryptology Conference, CRYPTO 2021, Virtual Event, August 16–20, 2021, Proceedings, Part II*, volume 12826 of *Lecture Notes in Computer Science*, pages 457–485. Springer, 2021.
- [15] Benedikt Bünz, Ben Fisch, and Alan Szeipieniec. Transparent snarks from dark compilers. In *Advances in Cryptology—EUROCRYPT 2020: 39th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Zagreb, Croatia, May 10–14, 2020, Proceedings, Part I* 39, pages 677–706. Springer, 2020.
- [16] Sai Sheshank Burra, Enrique Larraia, Jesper Buus Nielsen, Peter Sebastian Nordholt, Claudio Orlandi, Emmanuela Orsini, Peter Scholl, and Nigel P. Smart. High-performance multi-party computation for binary circuits based on oblivious transfer. *J. Cryptol.*, 34(3):34, 2021.
- [17] Matteo Campanelli, Antonio Faonio, Dario Fiore, Anaïs Querol, and Hadrián Rodríguez. Lunar: A toolbox for more efficient universal and updatable zkSNARKs and commit-and-prove extensions. In Mehdi Tibouchi and Huaxiong Wang, editors, *Advances in Cryptology - ASIACRYPT 2021 - 27th International Conference on the Theory and Application of Cryptology and Information Security, Singapore, December 6–10, 2021, Proceedings, Part III*, volume 13092 of *Lecture Notes in Computer Science*, pages 3–33. Springer, 2021.
- [18] Matteo Campanelli, Dario Fiore, and Anaïs Querol. Legosnark: Modular design and composition of succinct zero-knowledge proofs. In *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security*, pages 2075–2092, 2019.
- [19] Harsh Chaudhari, Ashish Choudhury, Arpita Patra, and Ajith Suresh. Astra: high throughput 3pc over rings with application to secure prediction. In *Proceedings of the 2019 ACM SIGSAC Conference on Cloud Computing Security Workshop*, pages 81–92, 2019.
- [20] David Chaum. Blind signature system. In *Advances in cryptology*, pages 153–153. Springer, 1984.
- [21] Ke Cheng, Yuheng Xia, Anxiao Song, Jiaxuan Fu, Wenjie Qu, Yulong Shen, and Jiaheng Zhang. Mosformer: Maliciously Secure Three-Party Inference Framework for Large Transformers, 2025. Publication info: Published elsewhere. ACM CCS 2025.
- [22] Koji Chida, Daniel Genkin, Koki Hamada, Dai Ikarashi, Ryo Kikuchi, Yehuda Lindell, and Ariel Nof. Fast large-scale honest-majority MPC for malicious adversaries. In Hovav Shacham and Alexandra Boldyreva, editors, *Advances in Cryptology - CRYPTO 2018 - 38th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 19–23, 2018, Proceedings, Part III*, volume 10993 of *Lecture Notes in Computer Science*, pages 34–64. Springer, 2018.
- [23] Ronald Cramer, Ivan Damgård, Daniel E. Escudero, Peter Scholl, and Chaoping Xing. SPDZ2k: Efficient MPC mod 2^k for dishonest majority. In *IACR Cryptology ePrint Archive*, 2018.
- [24] Anders Dalskov, Daniel Escudero, and Marcel Keller. Secure evaluation of quantized neural networks. *Proceedings on Privacy Enhancing Technologies*, 4:355–375, 2020.
- [25] Anders P. K. Dalskov, Daniel Escudero, and Marcel Keller. Secure evaluation of quantized neural networks. *Proc. Priv. Enhancing Technol.*, 2020(4):355–375, 2020.
- [26] Anders P. K. Dalskov, Daniel Escudero, and Marcel Keller. Fantastic four: Honest-majority four-party secure computation with malicious security. In Michael Bailey and Rachel Greenstadt, editors, *30th USENIX*

- Security Symposium, USENIX Security 2021, August 11-13, 2021*, pages 2183–2200. USENIX Association, 2021.
- [27] Ivan Damgård, Valerio Pastro, Nigel P. Smart, and Sarah Zakarias. Multiparty computation from somewhat homomorphic encryption. In Reihaneh Safavi-Naini and Ran Canetti, editors, *Advances in Cryptology - CRYPTO 2012 - 32nd Annual Cryptology Conference, Santa Barbara, CA, USA, August 19-23, 2012. Proceedings*, volume 7417 of *Lecture Notes in Computer Science*, pages 643–662. Springer, 2012.
 - [28] Ivan Damgård, Claudio Orlandi, and Mark Simkin. Black-box transformations from passive to covert security with public verifiability. Springer-Verlag, 2020.
 - [29] Leo de Castro and Anitgoni Polychroniadou. Lightweight, maliciously secure verifiable function secret sharing. In *Advances in Cryptology - EUROCRYPT 2022: 41st Annual International Conference on the Theory and Applications of Cryptographic Techniques, Trondheim, Norway, May 30 - June 3, 2022, Proceedings, Part I*, page 150–179, Berlin, Heidelberg, 2022. Springer-Verlag.
 - [30] Daniel Escudero, Satrajit Ghosh, Marcel Keller, Rahul Rachuri, and Peter Scholl. Improved primitives for MPC over mixed arithmetic-binary circuits. In Daniele Micciancio and Thomas Ristenpart, editors, *Advances in Cryptology - CRYPTO 2020 - 40th Annual International Cryptology Conference, CRYPTO 2020, Santa Barbara, CA, USA, August 17-21, 2020, Proceedings, Part II*, volume 12171 of *Lecture Notes in Computer Science*, pages 823–852. Springer, 2020.
 - [31] Sebastian Faust, Carmit Hazay, David Kretzler, and Benjamin Schlosser. Generic compiler for publicly verifiable covert multi-party computation. In Anne Canteaut and François-Xavier Standaert, editors, *Advances in Cryptology - EUROCRYPT 2021 - 40th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Zagreb, Croatia, October 17-21, 2021, Proceedings, Part II*, volume 12697 of *Lecture Notes in Computer Science*, pages 782–811. Springer, 2021.
 - [32] Jun Furukawa, Yehuda Lindell, Ariel Nof, and Or Weinstein. High-throughput secure three-party computation for malicious adversaries and an honest majority. In *Annual international conference on the theory and applications of cryptographic techniques*, pages 225–255. Springer, 2017.
 - [33] Chaya Ganesh, Vineet Nair, and Ashish Sharma. Dual polynomial commitment schemes and applications to commit-and-prove snarks. In Bo Luo, Xiaojing Liao, Jun Xu, Engin Kirda, and David Lie, editors, *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security, CCS 2024, Salt Lake City, UT, USA, October 14-18, 2024*, pages 884–898. ACM, 2024.
 - [34] Daniel Genkin, Yuval Ishai, and Antigoni Polychroniadou. Efficient multi-party computation: From passive to active security via secure SIMD circuits. In Rosario Gennaro and Matthew Robshaw, editors, *Advances in Cryptology - CRYPTO 2015 - 35th Annual Cryptology Conference, Santa Barbara, CA, USA, August 16-20, 2015, Proceedings, Part II*, volume 9216 of *Lecture Notes in Computer Science*, pages 721–741. Springer, 2015.
 - [35] Daniel Genkin, Yuval Ishai, Manoj Prabhakaran, Amit Sahai, and Eran Tromer. Circuits resilient to additive attacks with applications to secure computation. In David B. Shmoys, editor, *Symposium on Theory of Computing, STOC 2014, New York, NY, USA, May 31 - June 03, 2014*, pages 495–504. ACM, 2014.
 - [36] Niv Gilboa and Yuval Ishai. Distributed Point Functions and Their Applications. In David Hutchison, Takeo Kanade, Josef Kittler, Jon M. Kleinberg, Alfred Kobsa, Friedemann Mattern, John C. Mitchell, Moni Naor, Oscar Nierstrasz, C. Pandu Rangan, Bernhard Steffen, Demetri Terzopoulos, Doug Tygar, Gerhard Weikum, Phong Q. Nguyen, and Elisabeth Oswald, editors, *Advances in Cryptology - EUROCRYPT 2014*, volume 8441, pages 640–658. Springer Berlin Heidelberg, Berlin, Heidelberg, 2014. Series Title: Lecture Notes in Computer Science.
 - [37] Matan Hamilis, Elette Boyle, Niv Gilboa, Yuval Ishai, and Ariel Nof. Preprocessing for life: Dishonest-majority mpc with a trusted or untrusted dealer. In *2025 IEEE Symposium on Security and Privacy (SP)*, pages 41–41. IEEE Computer Society, 2024.
 - [38] Cheng Hong, Jonathan Katz, Vladimir Kolesnikov, Wen-jie Lu, and Xiao Wang. Covert security with public verifiability: Faster, leaner, and simpler. In Yuval Ishai and Vincent Rijmen, editors, *Advances in Cryptology - EUROCRYPT 2019 - 38th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Darmstadt, Germany, May 19-23, 2019, Proceedings, Part III*, volume 11478 of *Lecture Notes in Computer Science*, pages 97–121. Springer, 2019.
 - [39] Zhicong Huang, Wenjie Lu, Cheng Hong, and Jiansheng Ding. Cheetah: Lean and fast secure Two-Party deep neural network inference. In *31st USENIX Security Symposium (USENIX Security 22)*, pages 809–826, Boston, MA, August 2022. USENIX Association.
 - [40] Yuval Ishai, Manoj Prabhakaran, and Amit Sahai. Founding cryptography on oblivious transfer - efficiently. In *CRYPTO*, pages 572–591, 2008.
 - [41] Mitsuru Ito, Akira Saito, and Takao Nishizeki. Secret sharing scheme realizing general access structure. *Electronics and Communications in Japan (Part III: Fundamental Electronic Science)*, 72(9):56–64, 1989.
 - [42] Matthew Jagielski, Daniel Escudero, Rahul Rachuri, and Peter Scholl. Covert attacks on machine learning training in passively secure mpc. *arXiv preprint arXiv:2505.17092*, 2025.
 - [43] Marcel Keller. MP-SPDZ: A versatile framework for multi-party computation. In Jay Ligatti, Xinming Ou, Jonathan Katz, and Giovanni Vigna, editors, *CCS '20: 2020 ACM SIGSAC Conference on Computer and Communications Security, Virtual Event, USA, November 9-13, 2020*, pages 1575–1590. ACM, 2020.
 - [44] Marcel Keller and Ke Sun. Secure quantized training for deep learning. In Kamalika Chaudhuri, Stefanie Jegelka, Le Song, Csaba Szepesvári, Gang Niu, and Sivan Sabato, editors, *International Conference on Machine Learning, ICML 2022, 17-23 July 2022, Baltimore, Maryland, USA*, volume 162 of *Proceedings of Machine Learning Research*, pages 10912–10938. PMLR, 2022.
 - [45] Vladimir Kolesnikov and Alex J. Malozemoff. Public verifiability in the covert model (almost) for free. In *ASIACRYPT (2)*, pages 210–235. Springer, 2015.
 - [46] Nishat Koti, Mahak Pancholi, Arpita Patra, and Ajith Suresh. Swift: Super-fast and robust privacy-preserving machine learning. In *30th USENIX Security Symposium (USENIX Security 21)*, pages 2651–2668, 2021.
 - [47] Ryan Lehmkuhl, Pratyush Mishra, Akshayaram Srinivasan, and Raluca Ada Popa. Muse: Secure inference resilient to malicious clients. In *30th USENIX Security Symposium (USENIX Security 21)*, pages 2201–2218, 2021.
 - [48] Yun Li, Yufei Duan, Zhicong Huang, Cheng Hong, Chao Zhang, and Yifan Song. Efficient 3PC for binary circuits with application to Maliciously-Secure DNN inference. In *32nd USENIX Security Symposium (USENIX Security 23)*, pages 5377–5394, Anaheim, CA, August 2023. USENIX Association.
 - [49] Yun Li, Daniel Escudero, Yufei Duan, Zhicong Huang, Cheng Hong, Chao Zhang, and Yifan Song. Sublinear distributed product checks on replicated secret-shared data over $\mathbb{Z}_2^k$ without ring extensions. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security, CCS '24*, page 825–839, New York, NY, USA, 2024. Association for Computing Machinery.
 - [50] Yun Li, Cun Ye, Yuguang Hu, Ivring Morpheus, Yu Guo, Chao Zhang, Yupeng Zhang, Zhipeng Sun, Yiwen Lu, and Haodi Wang. Zkplus: Optimized fair-exchange protocol supporting practical and flexible data exchange. In Yongdae Kim, Jong Kim, Giovanni Vigna, and Elaine Shi, editors, *CCS '21: 2021 ACM SIGSAC Conference on Computer and Communications Security, Virtual Event, Republic of Korea, November 15 - 19, 2021*, pages 3002–3021. ACM, 2021.
 - [51] Yehuda Lindell, Eli Oxman, and Benny Pinkas. The ips compiler: Optimizations, variants and concrete efficiency. In *Advances in Cryptology - CRYPTO 2011 - 31st Annual Cryptology Conference*, volume 6841 of *Lecture Notes in Computer Science*, page 255. Springer, 2011.
 - [52] Helger Lipmaa. Prover-efficient commit-and-prove zero-knowledge snarks. *Int. J. Appl. Cryptogr.*, 3(4):344–362, 2017.
 - [53] Helger Lipmaa. On black-box knowledge-sound commit-and-prove snarks. In Jian Guo and Ron Steinfeld, editors, *Advances in Cryptology - ASIACRYPT 2023 - 29th International Conference on the Theory and Application of Cryptology and Information Security, Guangzhou, China, December 4-8, 2023, Proceedings, Part II*, volume 14439 of *Lecture Notes in Computer Science*, pages 41–76. Springer, 2023.
 - [54] Payman Mohassel and Peter Rindal. Aby^3 : A mixed protocol framework for machine learning. In David Lie, Mohammad Mannan, Michael Backes, and XiaoFeng Wang, editors, *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security, CCS 2018, Toronto, ON, Canada, October 15-19, 2018*, pages 35–52. ACM, 2018.
 - [55] Jesper Buus Nielsen, Peter Sebastian Nordholt, Claudio Orlandi, and Sai Sheshank Burra. A new approach to practical active-secure two-party computation. In Reihaneh Safavi-Naini and Ran Canetti, editors, *Advances in Cryptology - CRYPTO 2012 - 32nd Annual Cryptology Conference, Santa Barbara, CA, USA, August 19-23, 2012. Proceedings*, volume 7417 of *Lecture Notes in Computer Science*, pages 681–700. Springer, 2012.
 - [56] Arpita Patra and Ajith Suresh. BLAZE: blazing fast privacy-preserving machine learning. In *27th Annual Network and Distributed System*

Security Symposium, NDSS 2020, San Diego, California, USA, February 23-26, 2020. The Internet Society, 2020.

- [57] Allan B. Pedin and Nazli Siasi. Secure and decentralized anonymous e-voting scheme. In *Proceedings of the 2023 ACM Southeast Conference, ACMSE '23*, page 172–176, New York, NY, USA, 2023. Association for Computing Machinery.
- [58] Deevashwer Rathee, Mayank Rathee, Nishant Kumar, Nishanth Chandran, Divya Gupta, Aseem Rastogi, and Rahul Sharma. Cryptflow2: Practical 2-party secure inference. *IACR Cryptol. ePrint Arch.*, page 1002, 2020.
- [59] R. L. Rivest, A. Shamir, and D. A. Wagner. Time-lock puzzles and timed-release crypto. Technical report, USA, 1996.
- [60] Dragos Rotaru and Tim Wood. Marbled circuits: Mixing arithmetic and boolean circuits with active security. In Feng Hao, Sushmita Ruj, and Sourav Sen Gupta, editors, *Progress in Cryptology – INDOCRYPT 2019*, pages 227–249, Cham, 2019. Springer International Publishing.
- [61] Manuel B. Santos, Dimitris Mouris, Mehmet Ugurbil, Stanislaw Jarecki, José Reis, Shubho Sengupta, and Miguel de Vega. Curl: Private LLMs through Wavelet-Encoded Look-Up Tables. In *Proceedings of the Conference on Applied Machine Learning in Information Security*, pages 1–31, Arlington, Virginia, USA, October 24–25, 2024.
- [62] Peter Scholl, Mark Simkin, and Luisa Siniscalchi. Multiparty computation with covert security and public verifiability. In Dana Dachman-Soled, editor, *3rd Conference on Information-Theoretic Cryptography, ITC 2022, July 5-7, 2022, Cambridge, MA, USA*, volume 230 of *LIPIcs*, pages 8:1–8:13. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2022.
- [63] Adi Shamir. How to share a secret. *Communications of the ACM*, 22(11):612–613, 1979.
- [64] Ryan Wails, Aaron Johnson, Daniel Starin, Arkady Yerukhimovich, and S. Dov Gordon. Stormy: Statistics in tor by measuring securely. In *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security, CCS '19*, page 615–632, New York, NY, USA, 2019. Association for Computing Machinery.
- [65] Andrew C Yao. Protocols for secure computations. In *23rd annual symposium on foundations of computer science (sfcs 1982)*, pages 160–164. IEEE, 1982.

APPENDIX A COVERT SECURITY

Let $f : (\{0, 1\}^*)^n \rightarrow (\{0, 1\}^*)^n$ be an n -input function, and $\mathcal{F}_{\text{SECF}}$ is an ideal functionality that computes f with strong explicit cheat formulation. Let Corr be the set of corrupted parties controlled by $\mathcal{S}$. We now elaborate on the ideal execution in the following.

Inputs: Each honest party P_i sends its input x_i to $\mathcal{F}_{\text{SECF}}$. The ideal adversary $\mathcal{S}$ gets an auxiliary input z , and sends inputs on behalf of all corrupted parties to $\mathcal{F}_{\text{SECF}}$.

Abort Options: If a corrupted party P_i sends (abort, i) or $(\text{corrupted}, i)$ as its input to $\mathcal{F}_{\text{SECF}}$, $\mathcal{F}_{\text{SECF}}$ sends to (abort, i) or $(\text{corrupted}, i)$ respectively to all honest parties and halts. If multiple parties send the abort (or corrupted) message, $\mathcal{F}_{\text{SECF}}$ only keeps one of them. If both the two types of messages are received, $\mathcal{F}_{\text{SECF}}$ ignores the corrupted messages.

Attempted Cheat: If $\mathcal{S}$ sends (cheat, i) as the input of a corrupted party P_i , then $\mathcal{F}_{\text{SECF}}$ works as follows:

- **Detected:** With probability ϵ , $\mathcal{F}_{\text{SECF}}$ sends $(\text{corrupted}, i)$ to $\mathcal{S}$ and all the honest parties.
- **Undetected:** With probability $1 - \epsilon$, $\mathcal{F}_{\text{SECF}}$ sends undetected and the honest parties' inputs to $\mathcal{S}$. Then $\mathcal{S}$ specifies an output value y_i for each honest party P_i , and $\mathcal{F}_{\text{SECF}}$ hands each output y_i to P_i .

The ideal execution ends at this point. Otherwise, if no (abort, i) , $(\text{corrupted}, i)$ or (cheat, i) has been sent, the ideal execution continues as follows.

Functionality Answers Adversary: Let $\vec{x} := (x_1, \dots, x_n)$ be the received inputs by $\mathcal{F}_{\text{SECF}}$. The functionality computes the outputs $(y_1, \dots, y_n) := (x_1, \dots, x_n)$, and sends y_i to $\mathcal{S}$ for all $i \in \text{Corr}$.

Functionality Answers Honest Parties: The adversary $\mathcal{S}$ either sends back continue or (abort, i) w.r.t. a corrupted party P_i to $\mathcal{F}_{\text{SECF}}$. If it sends continue, $\mathcal{F}_{\text{SECF}}$ returns the output value y_i to each honest party P_i . Otherwise if it sends (abort, i) , then $\mathcal{F}_{\text{SECF}}$ hands (abort, i) to all honest parties.

Outputs: An honest party always outputs the message it received from $\mathcal{F}_{\text{SECF}}$. The corrupted parties output nothing. The adversary outputs an arbitrary probabilistic polynomial-time computable function of the initial inputs of the corrupted parties, the auxiliary input z , and the messages received from $\mathcal{F}_{\text{SECF}}$.

Let $\text{IDEAL}_{f, \mathcal{S}(z), \text{Corr}}^\epsilon(\vec{x}, 1^\kappa)$ be the outputs of the honest parties and $\mathcal{S}$ in an ideal execution. Let Π be a protocol that realizes the function f , which internally may use an auxiliary ideal functionality $\mathcal{G}$. Let $\text{REAL}_{\Pi, \mathcal{A}(z), \text{Corr}}^\epsilon(\vec{x}, 1^\kappa)$ be the outputs of the honest parties and $\mathcal{A}$ in a real execution. We now define the security of the protocol against covert adversaries with ϵ -deterrent.

Definition A.1: (Covert security with ϵ -deterrent.) Let f, Π, ϵ be as above. We say the protocol Π *securely computes f in the presence of a covert adversary with ϵ -deterrent* in the $\mathcal{G}$ -hybrid model if for any non-uniform probabilistic polynomial-time adversary $\mathcal{A}$ in the real world, there exists a probabilistic polynomial-time adversary $\mathcal{S}$ in the ideal world s.t. for every $\text{Corr} \subseteq [n]$, every balanced vector $\vec{x} \in (\{0, 1\}^*)^n$, and every auxiliary input $z \in \{0, 1\}^*$:

$$\{\text{IDEAL}_{f, \mathcal{S}(z), \text{Corr}}^\epsilon(\vec{x}, 1^\kappa)\}_{\kappa \in \mathbb{N}} \equiv_c \{\text{REAL}_{\Pi, \mathcal{A}(z), \text{Corr}}^\epsilon(\vec{x}, 1^\kappa)\}_{\kappa \in \mathbb{N}}.$$

Let $\text{OUTPUT}_j(\text{REAL}_{\Pi, \mathcal{A}(z), \text{Corr}}^\epsilon(\vec{x}, 1^\kappa))$ be the output of an honest party P_j in a real execution described above, and id_i be the identity of the party P_i . Let **Blame**, **Judge** be as described in Section III-B. We now introduce the formal definition of covert security with public verifiability below.

Definition A.2: (Covert security with ϵ -deterrent and public verifiability.) Let $f, \Pi, \epsilon, \text{Blame}, \text{Judge}$ be as above. We say $(\Pi, \text{Blame}, \text{Judge})$ *securely computes f in the presence of a covert adversary with ϵ -deterrent and public verifiability* in the $\mathcal{G}$ -hybrid model if the following conditions hold:

- 1) **Simulatability with ϵ -deterrence:** The protocol Π (where the honest parties broadcast cert if it detects cheating) is secure against a covert adversary with ϵ -deterrence factor in the $\mathcal{G}$ -hybrid model according to the strong explicit cheat formulation, but with a difference in the abort options: if a corrupted party sends (abort, i) to $\mathcal{F}_{\text{SECF}}$, the functionality sends abort to the honest parties and halts.
- 2) **Public Verifiability:** For any probabilistic polynomial-time adversary $\mathcal{A}$ corrupting parties P_i for $i \in \text{Corr} \subseteq [n]$, there exists a negligible function $\text{negl}(\kappa)$ s.t. for all $(\vec{x}, z) \in (\{0, 1\}^*)^{n+1}$ and $j \in [n] \setminus \text{Corr}$, the following holds: if

$$\text{OUTPUT}_j(\text{REAL}_{\Pi, \mathcal{A}(z), \text{Corr}}^\epsilon(\vec{x}, 1^\kappa)) = (\text{corrupted}, i),$$

then $\Pr[\text{Judge}(\text{cert}_j) = \text{id}_i] \geq 1 - \text{negl}(\kappa)$.

- 3) **Defamation Freeness:** For any probabilistic polynomial-time adversary $\mathcal{A}$ corrupting parties P_i for $i \in \text{Corr} \subseteq [n]$, there exists a negligible function $\text{negl}(\kappa)$ s.t. for all $(\vec{x}, z) \in (\{0, 1\}^*)^{n+1}$ and $j \in [n] \setminus \text{Corr}$,

$$\Pr[\text{Judge}(\text{cert}^*) = \text{id}_j \mid \text{cert}^* \leftarrow \mathcal{A}] \leq \text{negl}(\kappa).$$

APPENDIX B

SIGNATURES, COMMITMENTS, AND CP-NIZK PROOFS

A. Signature Schemes

A signature scheme consists of the following algorithms.

- $(\text{sk}, \text{pk}) \leftarrow \text{Gen}(1^\kappa)$: takes as input a security parameter, and generates a secret signing key sk and a public verification key pk .
- $\sigma \leftarrow \text{Sign}(\text{sk}; m)$: takes as input a message x , signs x with the signing key sk , and outputs a signature σ .
- $b \leftarrow \text{Vrfy}(\text{pk}; \sigma)$: verifies the validity of the signature with the verification key pk , and outputs $b \in \{0, 1\}$ indicating accept ($b = 1$) or reject ($b = 0$).

B. Commitments and Zero-knowledge Proofs

A commitment scheme [15] is a tuple of three algorithms $\text{COM} = (\text{Gen}, \text{Commit}, \text{Open})$ where

- $\text{pp} \leftarrow \text{Gen}(1^\kappa)$: takes as input a security parameter, and generates public parameters pp .
- $(\text{Com}, r) \leftarrow \text{Commit}(\text{pp}; x)$: takes as input x , produces a commitment Com and an opening hint r .
- $b \leftarrow \text{Open}(\text{pp}; \text{Com}, x, r)$: verifies the opening of commitment Com to x with the opening hint r , and outputs $b \in \{0, 1\}$ indicating accept ($b = 1$) or reject ($b = 0$).

We say a commitment scheme is (computationally) **binding** if for any probabilistic polynomial-time adversary $\mathcal{A}$,

$$\Pr \left[\begin{array}{l} b_0 = b_1 \neq 0 \\ \wedge \\ x_0 \neq x_1 \end{array} \middle| \begin{array}{l} \text{pp} \leftarrow \text{Gen}(1^\kappa) \\ (\text{Com}, x_0, x_1, \pi_0, \pi_1) \leftarrow \mathcal{A}(\text{pp}) \\ b_0 \leftarrow \text{Open}(\text{pp}, \text{Com}, x_0, \pi_0) \\ b_1 \leftarrow \text{Open}(\text{pp}, \text{Com}, x_1, \pi_1) \end{array} \right] \leq \text{negl}(\kappa).$$

We say a commitment is (perfectly) **hiding** if for any adversary $\mathcal{A}$,

$$\Pr \left[\mathcal{A}(\text{pp}, c) = b \middle| \begin{array}{l} \text{pp} \leftarrow \text{Gen}(1^\kappa), \\ (x_0, x_1) \leftarrow \mathcal{A}(\text{pp}), \\ b \xleftarrow{\$} \{0, 1\}, \\ (c, r) \leftarrow \text{Commit}(\text{pp}; x_b) \end{array} \right] = \frac{1}{2}.$$

A Non-Interactive Zero-Knowledge (NIZK) proof of knowledge is a tuple of algorithms $\text{ZK} = (\text{Gen}, \text{Prove}, \text{Vrfy})$:

- $\text{crs} \leftarrow \text{Gen}(1^\kappa)$: takes as input a security parameter, generates a common reference string crs .
- $\pi \leftarrow \text{Prove}(\text{crs}; s, w)$: takes as input a witness w and statement s , and outputs a proof π for $R(s, w) = 1$.
- $b \leftarrow \text{Vrfy}(\text{crs}; s, \pi)$: verifies the proof π on the statement s , and outputs $b \in \{0, 1\}$ indicating accept ($b = 1$) or reject ($b = 0$).

We say a zero-knowledge proof of knowledge has (perfect) **completeness**, if for any adversary $\mathcal{A}$ and for every $(s, w) \in R$,

$$\Pr \left[\text{Vrfy}(\text{crs}; s, \pi) = 1 \mid \begin{array}{l} \text{crs} \leftarrow \text{Gen}(1^\kappa), \\ \pi \leftarrow \text{Prove}(\text{crs}; s, w) \end{array} \right] = 1.$$

We say a zero-knowledge argument of knowledge has (computational) **knowledge soundness** if for any polynomial-time adversary $\mathcal{A}$, there exists a polynomial-time extractor $\mathcal{E}$ s.t.

$$\Pr \left[\begin{array}{l} (s, w) \notin R \wedge \\ \text{Vrfy}(\text{crs}; s, \pi) = 1 \end{array} \middle| \begin{array}{l} \text{crs} \leftarrow \text{Gen}(1^\kappa), \\ (s, \pi; w) \leftarrow (\mathcal{A} \parallel \mathcal{E})(\text{crs}) \end{array} \right] \leq \text{negl}(\kappa).$$

We say a zero-knowledge argument of knowledge has (perfect) **zero-knowledge** if for every pair of $(s, w) \in R$ and every adversary $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$ there exists a simulator $\mathcal{S}$ s.t.

$$\begin{aligned} & \Pr \left[\begin{array}{l} (s, w) \in R \wedge \\ \mathcal{A}_1(\text{crs}; s, \pi) = 1 \end{array} \middle| \begin{array}{l} \text{crs} \leftarrow \text{Gen}(1^\kappa), \\ (s, w) \leftarrow \mathcal{A}_2(\text{crs}), \\ \pi \leftarrow \text{Prove}(\text{crs}; s, w) \end{array} \right] \\ &= \Pr \left[\begin{array}{l} (s, w) \in R \wedge \\ \mathcal{A}_1(\text{crs}; s, \pi) = 1 \end{array} \middle| \begin{array}{l} \text{crs} \leftarrow \text{Gen}(1^\kappa), \\ (s, w) \leftarrow \mathcal{A}_2(\text{crs}), \\ \pi \leftarrow \mathcal{S}(\text{crs}; s) \end{array} \right]. \end{aligned}$$

APPENDIX C

ADDITIONAL RELATED WORKS

A. Function Secret Sharing(FSS)-based Approach

Function Secret Sharing (FSS) [10], [11] consists of two separate phases: key generation (Gen) and function evaluation (Eval). In the Gen phase, parties generate shared keys for the FSS scheme; in the Eval phase, they evaluate the function on these keys with their respective inputs. The distributed point function (DPF) [36] is a prominent instance of FSS, widely used to construct 2-server private information retrieval (PIR) and related cryptographic protocols.

Recent work has explored FSS-based approaches for malicious security in multi-party computation. Notably, Mosformer [21] proposes a maliciously secure three-party inference framework for secure comparison in large transformers, leveraging **verifiable** distributed point functions (VDPF) [29]. VDPF extends DPF with an additional verification procedure that uses zero-knowledge proofs to validate evaluation correctness. However, Mosformer does not report offline computation costs. Our evaluation reveals that VDPF key generation incurs significant computational overhead, making it impractical for certain applications. In contrast, our approach achieves substantially lower offline costs through covert security mechanisms.

B. G.O.D. in Maliciously Secure 3PC

Guaranteed Output Delivery (G.O.D.) is widely regarded as the strongest security guarantee achievable in maliciously secure computation. Previous works [12], [26], [46] achieve G.O.D. by identifying a conflicting pair of two parties (so that one of them must be corrupted), and treating the remaining party as a trusted party, who then receives inputs from all the other parties and computes the functionality in clear. In the general n -party setting, a similar approach is adopted in [13] by identifying conflicting pairs and then eliminating them from the computation, until the left parties obtain the correct outputs.

C. Covertly Secure Protocols

a) *The Cut-and-Choose Paradigm.*: The cut-and-choose paradigm is widely adopted in previous covertly secure protocols, where the parties need to run multiple, say, t instances of the semi-honest protocol in parallel, then randomly pick $t - 1$ instances and check if they are faithfully executed by opening all the inputs and random tapes of the parties. Note that this only applies to preprocessing protocols (where the parties have no private inputs, like the garbling phase or Beaver triple generation [8]), to avoid input

privacy leakage arising from opening. This cut-and-choose paradigm achieves a deterrence factor of $1 - \frac{1}{t}$. By setting $t = 2$, it can obtain a deterrence factor of 50%. However, to achieve a larger deterrence factor such as 80% or 90%, t will increase to 5 and 9 respectively, leading to notable communication and computational overhead of the semi-honest protocol.

b) Achieving Public Verifiability.: To achieve public verifiability based on the above cut-and-choose approach, previous works [62], [31], [28], [38], [45] use signatures to identify cheaters when checking fails. However, a cheating party may refuse to open its inputs and randomness by aborting the protocol after the random $t-1$ instances are chosen. This hinders the honest parties from generating a valid certificate of accusation. In the n -party setting, the state-of-the-art works [62], [31] introduce time-lock puzzles [59] to solve this issue, which allows a party to lock some data until a certain time has passed. In their compilers, the parties first compute puzzles to lock the shared coin used for sampling the $t-1$ instances and the random tapes used in those $t-1$ instances (which needs to be executed using a maliciously secure computation protocol, as the shared coin needs to be kept secret), and then sign the puzzles and broadcast the signatures. This ensures that even if the corrupted party aborts the protocol after the coin is made public, the honest parties are still able to obtain the random tapes of the corrupted party by solving its puzzles, and thus yield a valid public certificate of accusing the party. However, this approach incurs an unavoidable cost for computing the puzzles in a maliciously secure way, even in the optimistic case where no cheating is detected.

c) Beyond Preprocessing Protocols.: As aforementioned, the above cut-and-choose-based paradigm is only applicable to preprocessing protocols. Previous work [28], [31], [62] have shown that a general covertly secure computation protocol with public verifiability can be obtained by compiling the preprocessing protocol into a covertly secure one and then combining it with a maliciously secure online protocol. Additionally, previous works [51], [28], [31] have demonstrated another way to compile general input-dependent secure computation protocols based on the player virtualization paradigm [40]. Specifically, each real party now shares its private input to t virtual parties using a t -out-of- t secret sharing scheme, and simulates their executions of the semi-honest protocol. Rather than running t instances of a semi-honest n -party protocol, those virtual parties now run one instance of a semi-honest $(t \cdot n)$ -party protocol. To detect cheating, each party is required to randomly pick $t-1$ virtual parties and open their input shares as well as random tapes to check their behaviors. Note that privacy is now preserved due to the property of the secret sharing scheme. However, the scaled number of parties may blow up both the communication and computational costs when a large deterrence factor is required. For RSS-based protocol, this issue is more severe as the communication cost increases exponentially with the number of parties.

APPENDIX D

INSTANTIATIONS OF FUNCTIONALITIES

A. Instantiating $\mathcal{F}_{\text{input}}$

a) Input Sharing.: We have introduced the sharing semantics of ASTRA [19] in Section III-C. In Protocol D.1 we describe the protocol of ASTRA for instantiating $\mathcal{F}_{\text{input}}$, where a party shares a secret $x \in R$ to the other parties.

B. Instantiating $\mathcal{F}_{\text{VrfyIP}}$

We give the formal definition and instantiation of $\mathcal{F}_{\text{VrfyIP}}$ in Functionality C.1 and Protocol C.1. We apply the same optimizations from [49] to the protocol, and have the following Lemma from [49].

Lemma D.1: Let k, s, q be positive integers, and $T = 2^{\lceil \log_q m \rceil} + 1$. The protocol Π_{VrfyIP} securely computes $\mathcal{F}_{\text{VrfyIP}}$ in the $\mathcal{F}_{\text{coin}}$ -hybrid

Functionality C.1: $\mathcal{F}_{\text{VrfyIP}}$

Let $\mathcal{S}$ be the ideal world adversary.

- 1) $\mathcal{F}_{\text{VrfyIP}}$ receives the prover's identity j , a parameter m , and replicated secret shares of $\{(\llbracket \mu^{(i)} \rrbracket, \llbracket \vec{v}^{(i)} \rrbracket, \llbracket w^{(i)} \rrbracket)\}_{i=1}^m$ from the honest parties, and checks whether the shares are consistent. If not, $\mathcal{F}_{\text{CheckZero}}$ sends the identity j and honest parties' shares to $\mathcal{S}$, then receives an output $\text{out} \in \{\text{accept}, \text{abort}\}$ from $\mathcal{S}$, and passes the output to the honest parties. Otherwise, it reconstructs v the whole sharings $(\llbracket \mu^{(i)} \rrbracket, \llbracket \vec{v}^{(i)} \rrbracket, \llbracket w^{(i)} \rrbracket)$ for all $i \in [m]$, and sends the identity j and the shares of corrupted parties to $\mathcal{S}$. In addition, if P_j is corrupted, $\mathcal{F}_{\text{VrfyIP}}$ also sends the whole sharings $\{(\llbracket \mu^{(i)} \rrbracket, \llbracket \vec{v}^{(i)} \rrbracket, \llbracket w^{(i)} \rrbracket)\}_{i=1}^m$ to $\mathcal{S}$.
- 2) $\mathcal{F}_{\text{VrfyIP}}$ checks if the equation $w^{(i)} = \mu^{(i)} \cdot \vec{v}^{(i)}$ holds for all $i \in [m]$.
 - If it doesn't hold for some $i \in [m]$, $\mathcal{F}_{\text{VrfyIP}}$ sends abort to all honest parties and $\mathcal{S}$.
 - Otherwise, $\mathcal{F}_{\text{VrfyIP}}$ receives a command $\text{out} \in \{\text{accept}, \text{abort}\}$ from $\mathcal{S}$ and sends out to the honest parties.

model in the presence of a malicious adversary controlling one party, with soundness error bounded by

$$2^{-\lambda} + (1 - 2^{-\lambda}) \sum_{i=0}^s \frac{1}{2^{i+1}} \left(\sum_{j=0}^{T-2} \binom{s-i+j}{s-i} \cdot \frac{1}{2^{s-i+1+j}} \right)^2 + \frac{1}{2^{s+1}}.$$

APPENDIX E

SECURITY PROOF OF Cvt-MAC^{On}

We show the formal description of Cvt-MAC^{On} in Protocol D.2. We have the following theorem.

Theorem E.1: Let f be a three-party functionality over $\mathbb{Z}_{2^k}$. Let $s \in \mathbb{N}$ be an integer. Suppose PRG is a secure (pseudo-)random generator. Then Protocol D.2 securely computes f in the $(\mathcal{F}_{\text{coin}}, \mathcal{F}_{\text{VrfyIP}}, \mathcal{F}_{\text{checkzero}})$ -hybrid model in the presence of a covert adversary controlling one party with deterrence factor ϵ and abort where $\epsilon = 1 - \tau$ and τ is the soundness error of $\Pi_{\text{Cov}}^{\text{Abort}}$.

Proof E.1:

We first give the following lemma.

Lemma E.1: For any adversary $\mathcal{A}$ who corrupts P_t for $t \in \{1, 2\}$ and carries out additive attacks over $\mathbb{Z}_{2^k}$ in the online phase, or corrupts P_0 and carries out additive attacks when sharing authenticated randomness in the preprocessing phase of Protocol D.2, the honest parties will abort with soundness error bounded by $2^{-s+\log(s+1)}$.

Proof E.2: We first consider the former case. W.L.O.G., we assume that P_1 is corrupted. It may introduce an additive error $\varphi^{(i)}$ on $\delta_{z,1}^{(i)}$ when computing the i -th multiplication gate. We assume that $\varphi^{(i)} \neq 0$, as any additive error that equals zero over $\mathbb{Z}_{2^k}$ would do no harm to the output value. W.L.O.G., let $\vartheta^i \in \mathbb{Z}_{2^{k+s}}$ be an accumulated error on $x^{(i)}$ (i.e., $\delta_x^{(i)}$) from previous gates. We use $\text{val}(\llbracket x \rrbracket)^H$ (or $\text{val}(\llbracket x \rrbracket)^H$) to denote the value of the secret w.r.t. the honest parties' replicated (or additive, respectively) secret shares. Note that the error $\varphi^{(i)}$ will be passed into $\delta_z^{(i)}$, and thus $\text{val}(\llbracket z^{(i)} \rrbracket)^H = (x^{(i)} + \vartheta^i) \cdot y^{(i)} + \varphi^{(i)} = z^{(i)} + \vartheta^i y^{(i)} + \varphi^{(i)}$. After

Protocol C.1: Π_{VrfyIP} (Verifying Inner-Product Triples Known by a Single Party)

Input: The parties hold $\{(\llbracket \vec{a}_i \rrbracket, \llbracket \vec{b}_i \rrbracket, \llbracket c_i \rrbracket)\}_{i=1}^m$ over $\mathbb{Z}_{2^k}$ as input.

Auxiliary Input: Prover index j (where P_j knows the whole input sharings). Additionally, the parties agree on the parameters s, q and λ .

1) Random Linear Combination over $\mathbb{Z}_2$

- a) The parties invoke $\mathcal{F}_{\text{Coin}}$ to generate a random seed of size σ , and locally expand the seed and obtain random binary coefficients $\vec{\gamma}_1, \dots, \vec{\gamma}_\lambda \in \{0, 1\}^m$.
- b) For all $i \in [\lambda]$, all parties set $\llbracket \vec{\mu}_i \rrbracket = (\gamma_{i,1} \cdot \llbracket \vec{a}_1 \rrbracket, \dots, \gamma_{i,m} \cdot \llbracket \vec{a}_m \rrbracket)$, $\llbracket \vec{\nu}_i \rrbracket = (\llbracket \vec{b}_1 \rrbracket, \dots, \llbracket \vec{b}_m \rrbracket)$, $\llbracket w_i \rrbracket = \sum_{j=1}^m \gamma_{i,j} \cdot \llbracket c_j \rrbracket$.

2) Lifting Inner-Product Triples to $\mathbb{Z}_{2^{k+s}}$

For all $i \in [\lambda]$,

- a) The parties view $(\llbracket \vec{\mu}_i \rrbracket, \llbracket \vec{\nu}_i \rrbracket, \llbracket w_i \rrbracket)$ as replicated secret sharings over $\mathbb{Z}_{2^{k+s}}$. P_j computes $h_i :=_{k+s} \vec{\mu}_i' \cdot \vec{\nu}_i' - w_i'$, and additively shares $h_i/2^k$ over $\mathbb{Z}_{2^s}$.
- b) The parties locally multiply their shares of $\llbracket h_i/2^k \rrbracket$ by 2^k modulo 2^{k+s} , and obtain $\llbracket h_i \rrbracket$ over $\mathbb{Z}_{2^{k+s}}$.
- c) The parties set $\llbracket \tilde{w}_i' \rrbracket :=_{k+s} \llbracket w_i' \rrbracket + \llbracket h_i \rrbracket$.

3) Merging into One Inner-Product Triple

- a) The parties invoke $\mathcal{F}_{\text{Coin}}$ to sample random coefficients $\theta_1, \dots, \theta_\lambda \in \mathbb{Z}_{2^{k+s}}$.
- b) The parties set $\llbracket \vec{x} \rrbracket :=_{k+s} (\theta_1 \cdot \llbracket \vec{\mu}_1' \rrbracket, \dots, \theta_\lambda \cdot \llbracket \vec{\mu}_\lambda' \rrbracket)$, $\llbracket \vec{y} \rrbracket :=_{k+s} (\llbracket \vec{\nu}_1' \rrbracket, \dots, \llbracket \vec{\nu}_\lambda' \rrbracket)$ and $\llbracket z \rrbracket :=_{k+s} \sum_{i=1}^\lambda \theta_i \cdot \llbracket \tilde{w}_i' \rrbracket$.

4) Reducing the Dimension of the Inner-Product Triple

The parties repeat the following steps until the dimension of $(\llbracket \vec{x} \rrbracket, \llbracket \vec{y} \rrbracket, \llbracket z \rrbracket)$ is at most q .

- a) For all $i, i' \in [q]$ and $(i, i') \neq (1, 1)$, the prover party P_j computes $z_{i,i'} :=_{k+s} \vec{x}_i \cdot \vec{y}_{i'}$ and additively shares $z_{i,i'}$ to P_1, P_2 .
- b) The parties compute $\llbracket z_{1,1} \rrbracket$ by $\llbracket z_{1,1} \rrbracket :=_{k+s} \llbracket z \rrbracket - \sum_{i=2}^q z_{i,i}$.
- c) All parties invoke $\mathcal{F}_{\text{Coin}}$ to randomly sample $\{\alpha_i\}_{i=1}^q, \{\beta_i\}_{i=1}^q$ in $\mathbb{Z}_{2^{k+s}}$.
- d) The parties locally compute $\llbracket \vec{x}' \rrbracket :=_{k+s} \sum_{i=1}^q \alpha_i \cdot \llbracket \vec{x}_i \rrbracket$, $\llbracket \vec{y}' \rrbracket :=_{k+s} \sum_{i=1}^q \beta_i \cdot \llbracket \vec{y}_i \rrbracket$ and $\llbracket z' \rrbracket :=_{k+s} \sum_{i=1}^q \sum_{i'=1}^q \alpha_i \beta_{i'} \cdot \llbracket z_{i,i'} \rrbracket$.
- e) The parties set $(\llbracket \vec{x}' \rrbracket, \llbracket \vec{y}' \rrbracket, \llbracket z' \rrbracket) :=_{k+s} (\llbracket \vec{x}' \rrbracket, \llbracket \vec{y}' \rrbracket, \llbracket z' \rrbracket)$.

5) Checking the Final Multiplication Triple

The parties hold $(\llbracket \vec{x}' \rrbracket, \llbracket \vec{y}' \rrbracket, \llbracket z' \rrbracket)$ of dimension at most q .

- a) The prover party P_j randomly samples $x_0, y_0 \in \mathbb{Z}_{2^{k+s}}$. Then P_j additively shares x_0, y_0 to P_1, P_2 over $\mathbb{Z}_{2^{k+s}}$.
- b) For all $i, i' \in \{0, 1, \dots, q\}$ and $(i, i') \neq (1, 1)$, P_j computes $z_{i,i'} :=_{k+s} x_i \cdot y_{i'}$ and additively shares it to P_1, P_2 over $\mathbb{Z}_{2^k k + s}$.
- c) The parties compute $\llbracket z_{1,1} \rrbracket$ by $\llbracket z_{1,1} \rrbracket :=_{k+s} \llbracket z' \rrbracket - \sum_{i=2}^q z_{i,i}$. Note that P_j shares $z_{0,0} :=_{k+s} x_0 \cdot y_0$ to all parties in this step.
- d) The parties invoke $\mathcal{F}_{\text{Coin}}$ to randomly sample $\{\alpha_i\}_{i=1}^q, \{\beta_i\}_{i=1}^q$ in $\mathbb{Z}_{2^{k+s}}$. The parties set $\alpha_0 = \beta_0 = 1$.
- e) The parties locally compute $\llbracket x' \rrbracket :=_{k+s} \sum_{i=0}^q \alpha_i \cdot \llbracket x_i \rrbracket$, $\llbracket y' \rrbracket :=_{k+s} \sum_{i=0}^q \beta_i \cdot \llbracket y_i \rrbracket$ and $\llbracket z' \rrbracket :=_{k+s} \sum_{i=0}^q \sum_{i'=0}^q \alpha_i \beta_{i'} \cdot \llbracket z_{i,i'} \rrbracket$.
- f) The parties reconstruct $\llbracket x' \rrbracket, \llbracket y' \rrbracket, \llbracket z' \rrbracket$ and check whether $z' =_{k+s} x' \cdot y'$. If not, all parties abort. Otherwise, they output accept.

the random linear combination, we now have $\delta_w = \sum_{i=1}^m \eta^{(i)} \delta_z^{(i)} + \sum_{i=1}^m \eta^{(i)} (\vartheta^{(i)} y^{(i)} + \varphi^{(i)})$, and thus

$$\text{val}(\llbracket \tilde{w}' \rrbracket)^H = \tilde{w} + \alpha \cdot \sum_{i=1}^m \eta^{(i)} (\vartheta^{(i)} y^{(i)} + \varphi^{(i)}).$$

Similarly, we have $\llbracket \tilde{w} \rrbracket := \sum_{i=1}^m \eta^{(i)} ((\delta_x^{(i)} + \vartheta^{(i)}) \delta_y^{(i)} [\alpha] + (\delta_x^{(i)} + \vartheta^{(i)}) [\tilde{r}_y^{(i)}] + \delta_y^{(i)} [\tilde{r}_x^{(i)}]) + \llbracket \tilde{w}_0 \rrbracket = \tilde{w} + \sum_{i=1}^m \eta^{(i)} (\vartheta^{(i)} \delta_y^{(i)} \alpha + \vartheta^{(i)} \tilde{r}_y^{(i)})$. Given $\tilde{r}_y^{(i)} = \alpha \tilde{r}_y^{(i)}$, we have

$$\text{val}(\llbracket \tilde{w} \rrbracket)^H = \tilde{w} + \alpha \cdot \sum_{i=1}^m \eta^{(i)} \vartheta^{(i)} y^{(i)}.$$

Thus we have $\text{val}(\llbracket \tilde{w} - \tilde{w}' \rrbracket)^H = \alpha \sum_{i=1}^m \eta^{(i)} \varphi^{(i)}$. However, a malicious P_1 may introduce another error Δ when calling $\mathcal{F}_{\text{checkzero}}$, and thus

$$\text{val}(\llbracket o \rrbracket)^H = \alpha \sum_{i=1}^m \eta^{(i)} \varphi^{(i)} + \Delta.$$

Let $\Lambda := \sum_{i=1}^m \eta^{(i)} \varphi^{(i)}$, and $\Psi := -\Delta$. Then $\mathcal{F}_{\text{checkzero}}$ returns accept if and only if

$$\alpha \Lambda =_{k+s} \Psi. \quad (3)$$

Let $\text{pow}(x)$ be the largest power of two dividing x , and we define $\text{pow}(x) = k + s$ in the case that $x =_{k+s} 0$. We assume that there

Protocol D.1: Π_{Share} (Input Sharing).**Input:** x .**Auxiliary Input:** Party index j where P_j holds x .**Preprocessing:**

- 1) If $P_j = P_0$, P_0 and P_t for each $t \in \{1, 2\}$ run $r_{x,t} := \text{PRG}_{K_t}(\cdot)$.
- 2) If $P_j = P_t$ where $t \in \{1, 2\}$, P_0 and P_t run $r_{x,t} := \text{PRG}_{K_t}(\cdot)$, and then all the parties run $r_{x,3-t} := \text{PRG}_{K_t}(\cdot)$.
- 3) P_0 sets its share as $(r_{x,1}, r_{x,2})$.

Online:

- 1) P_j computes $\delta_x := x - (r_{x,1} + r_{x,2})$, and sends it to P_t for $t \in \{1, 2\}$ and $t \neq j$.
- 2) P_t for each $t \in \{1, 2\}$ sets its share of x as $(\delta_x, r_{x,t})$.

exists one $i_0 \in [m]$ s.t. $\varphi^{(i_0)} \neq_k 0$. Let E be the event that $\alpha\Lambda =_{k+s} \Psi$. We have the following claims.

- **Claim 1:** For any $k < j \leq k+s$, $\Pr[\text{pow}(\Lambda) = j] \leq 2^{-(j-k)}$. Note that $\Lambda =_{k+s} \sum_{i=1}^m \eta^{(i)} \varphi^{(i)}$, and the event $\text{pow}(\Lambda) = j$ indicates that $\text{pow}(\eta^{(i_0)}) = j - \text{pow}(\varphi^{(i_0)})$. As $\eta^{(i_0)}$ is uniformly random over $\mathbb{Z}_{2^{k+s}}$, the event that $\text{pow}(\eta^{(i_0)}) = j - \text{pow}(\varphi^{(i_0)})$ happens with probability $2^{-(j - \text{pow}(\varphi^{(i_0)}))}$. By assumption, we have $0 \leq \text{pow}(\varphi^{(i_0)}) \leq k$. Thus $\Pr[\text{pow}(\Lambda) = j] \leq 2^{-(j-k)}$.
- **Claim 2:** For any $k < j \leq k+s$, $\Pr[E | \text{pow}(\Lambda) = j] \leq 2^{-(k+s-j)}$. To see this, we now assume the event E holds given the condition $\text{pow}(\Lambda) = j$, i.e., $\alpha\Lambda =_{k+s} \Psi$ where $\text{pow}(\Lambda) = j$. We thus have $\alpha =_{k+s-j} \Psi \cdot (\frac{\Lambda}{2^j})^{-1}$ holds over ring 2^{k+s-j} . As α is uniformly random over $\mathbb{Z}_{2^{k+s}}$, this further indicates to decide the least $k+s-j$ bits of α , which happens with probability $2^{-(k+s-j)}$.
- **Claim 3:** For any $0 \leq j \leq k$, $\Pr[E | \text{pow}(\Lambda) = j] \leq 2^{-s}$. This follows the previous claim. Since now $0 \leq j \leq k$, we directly have $2^{-(k+s-j)} \leq 2^{-s}$.

Given the above claims, we now have

$$\begin{aligned}
\Pr[E] &= \Pr[E | 0 \leq \text{pow}(\Lambda) \leq k-1] \cdot \Pr[0 \leq \text{pow}(\Lambda) \leq k-1] \\
&\quad + \sum_{j=k+1}^{k+s} \Pr[E | \text{pow}(\Lambda) = j] \cdot \Pr[\text{pow}(\Lambda) = j] \\
&\leq 2^{-s} + s \cdot 2^{-(k+s-j)} \cdot 2^{-(j-k)} \\
&= 2^{-s + \log(s+1)}.
\end{aligned}$$

The case where P_0 is corrupted follows similarly, and the soundness error is also bounded by $2^{-s + \log(s+1)}$. $\square$

Let $\mathcal{S}$ be an ideal world adversary and $\mathcal{A}$ a real-world adversary controlling one corrupted party. In the beginning, $\mathcal{S}$ holds the corrupted party's input. We consider two different cases.

Case 1: P_0 is corrupted: $\mathcal{S}$ works as follows.

- 1) $\mathcal{S}$ first simulates the preprocessing step. $\mathcal{S}$ invokes $\mathcal{A}$ internally to get all the randomness used by P_0 , and receives shares of $r_{xy}^{(i)}$ from P_0 on behalf of the honest parties. Then $\mathcal{S}$ simulates $\mathcal{F}_{\text{vrfyIP}}^\epsilon$. $\mathcal{S}$ first recovers $r_x^{(i)}, r_y^{(i)}$ using these shares received from P_0 . Besides, it also recovers $r_{xy}^{(i)}$, denoted by $r'_{xy}^{(i)}$. Then it checks if $r'_{xy}^{(i)} =_k r_x^{(i)} \cdot r_y^{(i)}$ holds for all $i \in [m]$. If for some

$i \in [m]$ the equation doesn't hold, then with probability ϵ , $\mathcal{S}$ sends (corrupted, 0) to P_0 , and outputs whatever $\mathcal{A}$ outputs and terminates. Otherwise, $\mathcal{S}$ sends undetected to P_0 , and also sends the received shares of $r_x^{(i)}, r_y^{(i)}, r_{xy}^{(i)}$ to P_0 as the honest parties' inputs to $\mathcal{F}_{\text{vrfyIP}}^\epsilon$, and waits for P_0 to return a message as each honest party's output. Then $\mathcal{S}$ outputs whatever $\mathcal{A}$ outputs and terminates. If all the checks are passed, $\mathcal{S}$ simulates $\mathcal{F}_{\text{vrfyIP}}^\epsilon$ sending accept to the honest parties, and continues to receive shares of $\alpha, \tilde{r}_x^{(i)}, \tilde{r}_y^{(i)}$ as well as $\tilde{r}_w, \tilde{w}_0$ on behalf of the honest parties from $\mathcal{A}$, then proceeding to the next step.

- 2) $\mathcal{S}$ simulates the input sharing step. For each input wire, $\mathcal{S}$ emulates $\mathcal{F}_{\text{input}}$ and receives shares of this input held by P_0 .
- 3) $\mathcal{S}$ simulates the online verification step. $\mathcal{S}$ recovers α , and checks if P_0 cheated when sharing the authenticated randomness $\tilde{r}_x^{(i)}, \tilde{r}_y^{(i)}$ and also $\tilde{r}_w, \tilde{w}_0$ with α . If any check fails, $\mathcal{S}$ sends (cheat, 0) to the ideal functionality. If it receives detected from the ideal functionality, $\mathcal{S}$ simulates the honest parties outputting (corrupted, 0), and outputs whatever $\mathcal{A}$ outputs and terminates. Otherwise, if $\mathcal{S}$ receives undetected and the honest parties' inputs from the ideal functionality, it receives the outputs of the honest parties from the ideal functionality, which are delivered to the honest parties as outputs, then outputs whatever $\mathcal{A}$ outputs and terminates. Otherwise, if all the checks are passed, $\mathcal{A}$ simulates the honest parties proceeding to the next step.
- 4) $\mathcal{S}$ simulates the output reconstruction step. $\mathcal{S}$ receives the outputs of the corrupted party P_0 from the ideal functionality. For each output v , since $\mathcal{S}$ knows the shares $r_{v,1}, r_{v,2}$ held by P_0 of v , it computes $\delta_v :=_{k+s} v - r_{v,1} - r_{v,2}$, and sends it to P_0 on behalf of the honest parties. Then $\mathcal{S}$ outputs whatever $\mathcal{A}$ outputs and terminates.

Hybrid Arguments. We use the following hybrid arguments to show that the joint distribution of the views of the corrupted parties and the output of the honest parties are indistinguishable in the ideal world and the real world.

Hyb₁: In this hybrid, $\mathcal{S}$ uses the honest parties' inputs and honestly emulates the honest parties. This corresponds to the real world.

Hyb₂: In this hybrid, $\mathcal{S}$ runs Step (1) to simulate the preprocessing phase. The only difference between Hyb₂ and Hyb₁ is that now $\mathcal{S}$ simulates $\mathcal{F}_{\text{vrfyIP}}^\epsilon$ rather than the parties invoking it. Since $\mathcal{S}$ knows exactly the shares of honest parties received from P_0 , it can tell whether P_0 has cheated or not. If P_0 behaved honestly, then $\mathcal{S}$ simulate $\mathcal{F}_{\text{vrfyIP}}^\epsilon$ returning accept to the honest parties, which is the same as in Hyb₁. Otherwise, P_0 returns detected with probability ϵ . This behaves the same as $\mathcal{F}_{\text{vrfyIP}}^\epsilon$ in Hyb₁. Additionally, if the functionality returns undetected, since $\mathcal{S}$ knows the inputs of the honest parties (i.e., the received shares of the correlated randomness from P_0), it can return them to P_0 as $\mathcal{F}_{\text{vrfyIP}}^\epsilon$ does in Hyb₁. Thus, we have $\text{Hyb}_2 \equiv \text{Hyb}_1$.

Hyb₃: In this hybrid, $\mathcal{S}$ runs Step (3) to simulate the online verification phase. In Hyb₂, the honest parties validate the authenticated randomness $[\tilde{r}_x^{(i)}], [\tilde{r}_y^{(i)}], \tilde{r}_w$ and $\tilde{w}_0$ by directly opening α and applying a linear combination on the randomness using public coefficients from $\mathcal{F}_{\text{coin}}$, then calling $\mathcal{F}_{\text{checkzero}}$ on $[o']$. In Hyb₃, if $\mathcal{S}$ finds P_0 is cheating, it sends (cheat, 0) to the ideal functionality, who returns detected with probability ϵ . Note that after obtaining the return message, $\mathcal{S}$ can emulate the real-world execution of Hyb₂ via the public coefficients by simulating $\mathcal{F}_{\text{coin}}$ s.t. $o' \neq_{k+s} 0$ (when the functionality returns detected) or $o' =_{k+s} 0$ (when the functionality returns undetected). Hence, the distributions of the two hybrids are the same, and we have $\text{Hyb}_3 \equiv \text{Hyb}_2$.

Hyb₄: In this hybrid, $\mathcal{S}$ runs Step (4) to simulate the output reconstruction phase. Rather than honestly computing and sending the shares of the outputs of P_0 in Hyb₃, now $\mathcal{S}$ uses P_0 's outputs received from the ideal adversary, and as it knows the two additive shares of each output of P_0 , it can compute the shares that should be sent to P_0 for reconstruction, which are exactly the same as in Hyb₃. Thus, the two distributions are identical, and $\text{Hyb}_4 \equiv \text{Hyb}_3$.

Protocol D.2: $\Pi_{\text{Cvt-MAC}^{\text{on}}}$ (Securely Computing an Arithmetic Circuit over Ring $\mathbb{Z}_{2^k}$ with Abort)

Input: Each party P_j where $j \in \{0, 1, 2\}$ holds inputs $v_j^{(i)} \in \mathbb{Z}_{2^k}$ where $i \in [d]$.

Auxiliary Input: The parties hold a description of an arithmetic circuit C over $\mathbb{Z}_{2^k}$ on inputs of length d . Let m be the number of multiplication gates in C . The parties agree on a parameter $s \in \mathbb{N}$.

Preprocessing:**1) Circuit Preprocessing**

- a) For each input wire w.r.t the input $v_j^{(i)}$, the parties run the preprocessing step of Π_{Share} on $v_j^{(i)}$ over $\mathbb{Z}_{2^{k+s}}$.
- b) For each gate g in a predetermined topological order, if g is a multiplication (or addition) gate, the parties run the preprocessing step of Π_{Mul} on the two input sharings (or P_0 locally add its shares, respectively) over $\mathbb{Z}_{2^{k+s}}$.

2) Verification Preprocessing

- a) Denote by $\{[r_x^{(i)}], [r_y^{(i)}], [r_{xy}^{(i)}]\}_{i=1}^m$ the m triples produced in the circuit preprocessing step. The parties call $\mathcal{F}_{\text{VrfyIP}}$ on these triple to check if $r_x^{(i)} \cdot r_y^{(i)} =_k r_{xy}^{(i)}$ for each $i \in [m]$. If an abort is received, the parties abort and then halt, otherwise they proceed to the next step.
- b) P_0 uniformly picks a randomness α from $\mathbb{Z}_{2^{k+s}}$, and computes $\tilde{r}_x^{(i)} :=_{k+s} \alpha \cdot r_x^{(i)}$, $\tilde{r}_y^{(i)} :=_{k+s} \alpha \cdot r_y^{(i)}$. Then P_0 additively shares α and $\{\tilde{r}_x^{(i)}\}_{i=1}^m, \{\tilde{r}_y^{(i)}\}_{i=1}^m$ to P_1, P_2 .
- c) P_0 random picks a seed $\text{sd} \in \{0, 1\}^\kappa$, and additively shares it to P_1, P_2 along with the seed's commitment. P_0 run PRG with sd to derive m randomness $\{\eta^{(i)}\}_{i=1}^m$, and computes $\tilde{w}_0 := \sum_{i=1}^m \eta^{(i)} \tilde{r}_{xy}^{(i)}$. Let $\{r_z^{(i)}\}_{i=1}^m$ be the randomnesses prepared for the outputs of the m multiplication gates. P_0 computes $\tilde{r}_w := \alpha \cdot \sum_{i=1}^m \eta^{(i)} \cdot r_z^{(i)}$, and additively shares $\tilde{w}_0, \tilde{r}_w$ to P_1, P_2 .

Online:**1) Input Sharing**

For each input wire w.r.t the input $v_j^{(i)}$, the parties run the online step of Π_{Share} on $v_j^{(i)}$ over $\mathbb{Z}_{2^{k+s}}$.

2) Circuit Evaluation

For each gate g in the predetermined topological order, if g is a multiplication (or addition) gate, the parties run the online step of Π_{Mul} on the two input sharings to g (or P_1, P_2 locally add their shares, respectively) over $\mathbb{Z}_{2^{k+s}}$.

3) Verification

Let $\{[z^{(i)}]\}_{i=1}^m$ be the outputs of the m multiplication gates generated in the previous step.

- a) P_1, P_2 compute $[\tilde{w}] := \sum_{i=1}^m \eta^{(i)} (\delta_x^{(i)} \delta_y^{(i)} [\alpha] + \delta_x^{(i)} [\tilde{r}_y^{(i)}] + \delta_y^{(i)} [\tilde{r}_x^{(i)}]) + [\tilde{w}_0]$.
- b) P_1, P_2 reconstruct sd , check the commitment, and derive m randomness $\{\eta^{(i)}\}_{i=1}^m$ using PRG with sd . Then P_1, P_2 compute $\delta_w := \sum_{i=1}^m \eta^{(i)} \delta_z^{(i)}$, and $[\tilde{w}'] := \delta_w [\alpha] + [\tilde{r}_w]$.
- c) P_1, P_2 compute $[o] :=_{k+s} [\tilde{w}] - [\tilde{w}']$, and then call $\mathcal{F}_{\text{checkzero}}$ on $[o]$ to check if $o =_{k+s} 0$. If an reject is received, P_1, P_2 broadcast abort and then halt.
- d) P_1, P_2 reconstruct α , and call $\mathcal{F}_{\text{coin}}$ to obtain $2m + 2$ randomness $\{\xi^{(i)}\}_{i=1}^{2m+2}$. P_1, P_2 compute $[o'] := \sum_{i=1}^m \xi^{(i)} (\alpha[r_x^{(i)}] - [\tilde{r}_x^{(i)}]) + \sum_{i=1}^m \xi^{(m+i)} (\alpha[r_y^{(i)}] - [\tilde{r}_y^{(i)}]) + \xi^{(2m+1)} (\alpha \sum_{i=1}^m \eta^{(i)} [r_{xy}^{(i)}] - [\tilde{w}_0]) + \xi^{(2m+2)} (\alpha[r_w] - [\tilde{r}_w])$, and then call $\mathcal{F}_{\text{checkzero}}$ on $[o']$. If any reject is received, P_1, P_2 broadcast abort and then halt.

4) Output Reconstruction

If no abort command is received in all the previous steps, for every output gate corresponding to some party P_j , the parties run Rec to reconstruct the output value to P_j . Otherwise, they broadcast abort to the other parties and halt.

Hyb₅: In this hybrid, $\mathcal{S}$ runs Step (2) to simulate the input sharing phase. Compared to Hyb₅ where the parties invoke $\mathcal{F}_{\text{input}}$ to share their inputs as in the real world, now $\mathcal{S}$ emulates $\mathcal{F}_{\text{input}}$ by sending random values as the honest parties' shares to P_0 . Due to the secrecy property of the secret sharing scheme and the use of PRG, the two distributions are (computationally) indistinguishable, and we have $\text{Hyb}_5 \equiv_c \text{Hyb}_4$.

Note that Hyb₅ corresponds to the ideal world scenario, and in all the above hybrids, the joint distribution of the views of the adversary and the output of the honest parties remains unchanged. This shows that the two distributions in the ideal world and the real world are (computationally) indistinguishable.

Case 2: P_1 or P_2 is corrupted: W.L.O.G., we assume P_1 is the corrupted party. The ideal world adversary $\mathcal{S}$ works as follows.

- 1) $\mathcal{S}$ simulates the preprocessing step. $\mathcal{S}$ invokes $\mathcal{A}$ internally to get all the random shares of $r_x^{(i)}, r_y^{(i)}, r_z^{(i)}$ held by P_1 . $\mathcal{S}$ plays the role of P_0 to share $r_{xy}^{(i)}$ by sending random values to P_1 . Then $\mathcal{S}$ simulates $\mathcal{F}_{\text{VrfyIP}}^\epsilon$. As it knows all the additive shares of $r_x^{(i)}, r_y^{(i)}$ and $r_{xy}^{(i)}$ held by P_1 , $\mathcal{S}$ checks the correctness of the messages sent from P_1 to $\mathcal{F}_{\text{VrfyIP}}^\epsilon$. If they are incorrect, $\mathcal{S}$ knows that P_1 has cheated; it then returns detected to P_1 with probability ϵ , outputs whatever P_1 outputs and then terminates. Otherwise, $\mathcal{S}$ returns undetected as well as valid shares of some

random triples $(r_x^{(i)}, r_y^{(i)}, r_{xy}^{(i)})$ (w.r.t. P_1 's shares) as the honest parties' inputs to P_1 , and then receives a message from P_1 as each honest party's output. Then S outputs whatever P_1 outputs and then terminates. Otherwise, if P_1 is not cheating, S sends random values to P_1 as its shares of $\alpha \tilde{r}_x^{(i)}, \tilde{r}_y^{(i)}$ (or invokes $\mathcal{A}$ internally to get its random shares of $\tilde{r}_x^{(i)}, \tilde{r}_y^{(i)}$).

- 2) S simulates the input sharing step. For each input wire, S emulates $\mathcal{F}_{\text{input}}$ and receives shares of this input held by P_1 .
- 3) S simulates the circuit evaluation step. For each multiplication gate, S sends a random value to P_1 as $\delta_{z,2}^{(i)}$ on behalf of P_2 .
- 4) S simulates the online verification step. As S knows all the inputs and randomness of P_1 , as well as all the messages sent to P_1 by itself during the previous step, it can tell if P_1 has cheated by checking each $\delta_{z,1}^{(i)}$. If P_1 is not cheating, S sends a random value as sd_2 on behalf of P_2 , and then simulate $\mathcal{F}_{\text{checkzero}}$ returning accept to P_1 . Otherwise, S sends (cheat, 1) to the ideal functionality. If it receives detected from the ideal functionality, S picks sd_2 s.t. the derived randomness from the recovered sd satisfy that $\sum_{i=1}^m \eta^{(i)} \varphi^{(i)} \neq_{k+s} 0$, and sends sd_2 to P_1 on behalf of P_2 . Then it simulates the honest parties outputting abort, then outputs whatever $\mathcal{A}$ outputs and terminates. Otherwise, if S receives undetected and the honest parties' inputs from the ideal functionality, it picks sd_2 s.t. the derived randomness from the recovered sd satisfy that $\sum_{i=1}^m \eta^{(i)} \varphi^{(i)}$ has an inverse over $\mathbb{Z}_{2^{k+s}}$. Then it receives the share of o from P_1 as the input of $\mathcal{F}_{\text{checkzero}}$, then computes the potential additive error Δ (that is the difference between the actually received share from P_1 and the one that should be sent from P_1). Next S returns accept to P_1 on behalf of $\mathcal{F}_{\text{checkzero}}$, then computes $\alpha :=_{k+s} -\Delta \cdot (\sum_{i=1}^m \eta^{(i)} \varphi^{(i)})^{-1}$, and obtains α_2 from the computed α and the share of α held by P_1 . S sends α_2 to P_1 on behalf of P_2 . Then it simulates $\mathcal{F}_{\text{coin}}$ sending public coefficients to P_1 , and checks whether the share of o' sent by P_1 when calling $\mathcal{F}_{\text{checkzero}}$ is correct or not. If not, S emulates $\mathcal{F}_{\text{checkzero}}$ sending reject to P_1 , otherwise it sends accept.
- 5) S simulates the output reconstruction step. S receives the outputs of the corrupted party P_1 from the ideal functionality. As S knows the inputs and randomness as well as all the received messages of P_1 , for each output v , S calculates the shares $\delta_v, r_{v,t}$ held by P_1 of v . Then it further computes $r_{v,2} :=_{k+s} v - \delta_v - r_{v,t}$, and sends it to P_0 on behalf of the honest parties.
- 6) S simulates the cheater identification step as in Case 1.

Hybrid Arguments. We use the following hybrid arguments to show that the joint distribution of the views of the corrupted parties and the output of the honest parties are indistinguishable in the ideal world and the real world.

Hyb₁: In this hybrid, S uses the honest parties' inputs and honestly emulates the honest parties. This corresponds to the real world.

Hyb₂: In this hybrid, S runs Step (1) to simulate the preprocessing phase. The only difference between Hyb₂ and Hyb₁ is that now S simulates $\mathcal{F}_{\text{VrfyIP}}^\epsilon$ rather than the parties invoking it. Since S knows exactly the shares held by P_1 , it can tell whether P_1 has cheated or not. If P_1 behaved honestly, then S sends accept to P_1 , which is the same as in the real world. Otherwise, it returns detected with probability ϵ . This behaves the same as $\mathcal{F}_{\text{VrfyIP}}$ in the real world. Additionally, if S returns undetected, then it can return valid randomness and the corresponding shares as honest P_0 and P_2 's inputs as $\mathcal{F}_{\text{VrfyIP}}^\epsilon$ does in the real world. Thus, this makes no difference to $\mathcal{A}$ both in the ideal world and the real world, and we have $\text{Hyb}_2 \equiv \text{Hyb}_1$.

Hyb₃: In this hybrid, S runs Step (3) to simulate the circuit evaluation phase. The only difference between Hyb₃ and Hyb₂ is that S sends a random value rather than an honestly computed share $\delta_{z,2}$ to P_1 . Since $r_{z,2}$ is a (pseudo-)random value generated by PRG and in Hyb₂, $\delta_{z,2}$ is masked by $r_{z,2}$, the views of $\mathcal{A}$ in two hybrids are (computationally) indistinguishable. Thus we have $\text{Hyb}_3 \equiv_c \text{Hyb}_2$.

Hyb₄: In this hybrid, S runs Step (4) to simulate the online verification phase. In Hyb₄, S simulates $\mathcal{F}_{\text{checkzero}}$ rather than the parties invoking it in Hyb₃. As S can exactly compute the additive error introduced by P_1 , it can tell whether P_1 has cheated or not. If it detects cheating, it queries the ideal functionality and returns detected with probability ϵ . If the cheating is detected by the ideal functionality, then S chooses the seed accordingly s.t. the accumulated error doesn't vanish over $\mathbb{Z}_{2^{k+s}}$. Otherwise, S chooses the seed as well as α accordingly s.t. the accumulated error vanishes over $\mathbb{Z}_{2^{k+s}}$. This corresponds to the real-world execution. Thus we have $\text{Hyb}_4 \equiv \text{Hyb}_3$.

Hyb₅: In this hybrid, S runs Step 5 to simulate the output reconstruction phase. For each output of P_1 , S can first compute the two additive shares of this output held by P_1 using its inputs and views. Then using the output value that S receives from the ideal functionality, S can calculate the additive share that should be sent from the honest parties to P_1 , which are exactly the same as in Hyb₄. Thus, the two distributions are identical, and $\text{Hyb}_5 \equiv \text{Hyb}_4$.

Hyb₆: In this hybrid, S runs Step 2 to simulate the input sharing step as in Case 1. Analogously, we have $\text{Hyb}_6 \equiv_c \text{Hyb}_5$.

Hyb₇: In this hybrid, the honest parties use the outputs received from the functionality as in Case 1. Analogously, we have $\text{Hyb}_7 \equiv \text{Hyb}_6$.

Note that Hyb₇ corresponds to the ideal world scenario, and in all the above hybrids, the joint distribution of the views of the adversary and the output of the honest parties remains unchanged. This shows that the two distributions in the ideal world and the real world are (computationally) indistinguishable. This concludes our proof.

APPENDIX F SOUNDNESS ANALYSIS OF PROTOCOL E.1

W.L.O.G., we instantiate the protocol using the RSS scheme. We denote the RSS-based input sharing, multiplication, and addition protocol by Π_{RShare} , Π_{RMul} , and Π_{RAdd} , respectively, and give a formal description of our covertly secure protocol for binary circuits in Protocol E.1 (we omit the cheater identification procedure here for simplicity). We give a soundness analysis of Protocol E.1 below.

Let $t > 0$ be the number of bad triples corrupted by $\mathcal{A}$ among the initial $M + 1$ semi-honest random triples, and let T be the number of buckets checked in the sacrifice procedure. First, we define some events:

- Event $E(M, T, t)$: with parameter M, T, t described before, the $\mathcal{A}$ passes the triple verification procedure.
- Event $E_0(M, t)$: randomly pick one triple from the $M + 1$ triples, which is a correct triple.
- Event $E_1(M, T, t, j)$: randomly select $2T$ triples from the rest M triples, where there exactly includes j incorrect triples. Obviously, $\max(0, t - N + 2T) \leq j \leq \min(2T, t)$. We denote the lower bound and upper bound as j_0, j_n , respectively.
- Event $E_2(T, j)$: if there are j incorrect triples in the $2T$ triples, after randomly packing them into T buckets, each of the buckets contains a pair of identical triples.

We denote the possibility of event E_i as P_{E_i} . Since N and T are fixed parameters in this lemma, we use the simplified notation $E(t), E_0(t), E_1(t, j), E_2(j)$ to refer to these events for brevity.

It is clear that

$$P_E(t) = P_{E_0}(t) \cdot \sum_{j=j_0}^{j_n} [P_{E_1}(t, j) \cdot P_{E_2}(j)].$$

For a special case $t = 1$, the event E can only occur via a single path: the incorrect triple must be excluded both from the random openings and from the set of T buckets selected for sacrifice. If the unique incorrect triple is included in the $2T$ selection, it is inevitably paired with a correct triple in some bucket, causing the check to fail. So

$$P_E(1) = \frac{M}{M+1} \frac{M-2T}{M} \leq 1 - \frac{2T}{M}. \quad (4)$$

Protocol E.1: $\Pi_{\text{Cvt-Triple}^{\text{On}}}$ (Securely Computing Binary Circuits with Abort)

Input: Each party P_j where $j \in \{0, 1, 2\}$ holds inputs $v_j^{(i)} \in \mathbb{Z}_2$ where $i \in [d]$.

Auxiliary Input: The parties hold a description of an binary circuit C over $\mathbb{Z}_2$ on inputs of length d . Let N be the number of AND gates in C . The parties agree on parameters M and T s.t. $M - T = N$.

Preprocessing:

1) **Generate Random Triples**

- a) The parties locally generate $M + 1$ pairs $(\llbracket a^{(i)} \rrbracket, \llbracket b^{(i)} \rrbracket)$ using PRG with shared keys for $i \in [M + 1]$, and run the multiplication protocol to obtain $\llbracket c^{(i)} \rrbracket := \llbracket a^{(i)} \rrbracket \cdot \llbracket b^{(i)} \rrbracket$.

2) **Cut-and-Bucket**

- a) The parties randomly shuffle the random $M + 1$ triples, and open the first triple to check its correctness.
b) If the opened triple is correct, the parties divide the first $2T$ unopened triples into T buckets, each of size 2, and verify the first triple with the second one in each triple.
c) If all the checks are passed, the parties outputs the left unchecked $M - 2T$ triples and all the first triples in these T buckets.

Online:

1) **Input Sharing**

The parties call $\mathcal{F}_{\text{input}}$ to share their inputs.

2) **Circuit Evaluation**

Let $\{\llbracket a^{(i)} \rrbracket, \llbracket b^{(i)} \rrbracket, \llbracket c^{(i)} \rrbracket\}_{i=1}^N$ be the prepared AND triples. For each gate g in the predetermined topological order:

- a) If g is an addition gate, the parties locally add the two input sharings.
b) If g is an AND gate (w.r.t. index i) with input sharings $\llbracket x^{(i)} \rrbracket$ and $\llbracket y^{(i)} \rrbracket$, the parties compute $\rho^{(i)} := x^{(i)} \oplus a^{(i)}$ and $\sigma^{(i)} := y^{(i)} \oplus b^{(i)}$. Then they run Rec to open $\rho^{(i)}, \sigma^{(i)}$, and locally compute $\llbracket z^{(i)} \rrbracket := \rho^{(i)} \cdot \sigma^{(i)} \oplus \rho^{(i)} \cdot \llbracket b^{(i)} \rrbracket \oplus \sigma \cdot \llbracket a^{(i)} \rrbracket \oplus \llbracket c^{(i)} \rrbracket$.

3) **Output Reconstruction:**

If no abort command is received in all the previous steps, for every output gate corresponding to some party P_j , the parties run Rec to reconstruct the output value to P_j . Otherwise, they broadcast abort to the other parties and halt.

For another special case $t = M$, we have $P_E = P_{E_0}$ since there will always be all incorrect triples in the T buckets and will always pass the test. So

$$P_E(M) = \frac{1}{M+1}. \quad (5)$$

When $2 \leq t < M$, note that

$$\begin{aligned} P_E(t) &= P_{E_0}(t) \cdot \sum_{j=j_0}^{j_n} [P_{E_1}(t, j) \cdot P_{E_2}(j)] \\ &= P_{E_0}(t) \cdot \sum_{j=0}^{2T} [P_{E_1}(t, j) \cdot P_{E_2}(j)] \\ &= P_{E_0}(t) \cdot \sum_{j=2}^{2T-2} [P_{E_1}(t, j) \cdot P_{E_2}(j)] \\ &\quad + P_{E_0}(t) \cdot [P_{E_1}(t, 0) + P_{E_1}(t, T)] \\ &< P_{E_0}(t) \cdot \max_{j=2}^{2T-2} \{P_{E_2}(j)\} \sum_{j=2}^{2T-2} [P_{E_1}(t, j)] \\ &\quad + P_{E_0}(t) \cdot [P_{E_1}(t, 0) + P_{E_1}(t, T)]. \end{aligned}$$

The probability P_{E_1} follows a hypergeometric distribution. Since the summation only includes a subset of the terms from this distribution,

its total value must be at most 1. So

$$\begin{aligned} P_E(t) &< P_{E_0}(t) \cdot \max_{j=2}^{2T-2} \{P_{E_2}(j)\} \\ &\quad + P_{E_0}(t) \cdot [P_{E_1}(t, 0) + P_{E_1}(t, T)]. \end{aligned}$$

To analyze $P_{E_2}(j)$, recall that a successful outcome requires all incorrect triples to be perfectly paired and placed into buckets. We restrict our attention to even values of j , since an odd number of incorrect triples would inevitably leave one unpaired, causing the check to fail. The number of successful pairings is $\binom{T}{j/2}$, while the total number of ways to choose j triples out of $2T$ is $\binom{2T}{j}$. Therefore, the probability is given by

$$P_{E_2}(j) = \frac{\binom{T}{j/2}}{\binom{2T}{j}}.$$

It is not hard to show that this sequence achieves its maximum at the boundary. In particular, for $2 \leq j \leq 2T - 2$, the maximum value of $P_{E_2}(j)$ is attained at $j = 2$, where $P_{E_2}(2) = \frac{1}{2T-1}$. So

$$\begin{aligned} P_E(t) &< \frac{M+1-t}{(M+1)(2T-1)} + P_{E_0}(t) \cdot [P_{E_1}(t, 0) + P_{E_1}(t, T)] \\ &< \frac{M-1}{(M+1)(2T-1)} + P_{E_0}(t) \cdot [P_{E_1}(t, 0) + P_{E_1}(t, T)]. \end{aligned} \quad (6)$$

We now analyze the remaining term $P_{E_0}(t)P_{E_1}(t, 0)$. Similar to the earlier case, we can study the ratio between successive terms—specifically, for t and $t+1$ —to show that this sequence is

monotonically decreasing w.r.t. t . Therefore, its maximum is achieved at $t = 2$, yielding the bound

$$P_{E_0}(t)P_{E_1}(t, 0) \leq \frac{(M - 2T)(M - 2T - 1)}{M(M + 1)}. \quad (7)$$

Finally, we analyze the remaining term $P_{E_0}(t)P_{E_1}(t, T)$. Directly analyzing this product is challenging, so we upper-bound each factor individually. Using the same ratio-based argument as before, we find that $P_{E_1}(t, T)$ attains its maximum at $t = M - 1$, with

$$P_{E_1}(t, T) \leq \frac{M - 1 - 2T}{M}.$$

Note that the term $P_{E_0}(t)P_{E_1}(t, T)$ corresponds to the case where $j = 2T$, which implies that $j_n = \min(2T, t) = 2T$, so we must have $t > 2T$. Under this condition, we can bound the first factor by

$$P_{E_0}(t) < \frac{M + 1 - 2T}{M + 1}.$$

Combining the two bounds, we obtain:

$$P_{E_0}(t)P_{E_1}(t, T) \leq \frac{(M - 1 - 2T)(M + 1 - 2T)}{M(M + 1)}. \quad (8)$$

Putting inequalities (6) (7) (8) together, for $2 \leq t \leq M - 1$, we have

$$\begin{aligned} P_E(t) &< \frac{M - 1}{(M + 1)(2T - 1)} \\ &+ \frac{(M - 2T)(M - 2T - 1)}{M(M + 1)} \\ &+ \frac{(M - 1 - 2T)(M + 1 - 2T)}{M(M + 1)}. \end{aligned} \quad (9)$$

For all $1 \leq t \leq M$, the soundness error is bounded by the maximum of upper bounds (4), (5) and (9).

Denote T to be $\frac{\epsilon_0 M}{2}$, inequality (9) could be written as

$$P_E(t) < \frac{2(1 - \epsilon_0)^2 M}{M + 1} + O\left(\frac{1}{M}\right) < 2(1 - \epsilon_0)^2 + O\left(\frac{1}{M}\right).$$

The remaining term is negligible when M is large enough. For deterrent factors $\epsilon \geq 0.5$, we may initially set $\epsilon_0 = \epsilon$, which yields $2(1 - \epsilon_0)^2 < (1 - \epsilon)$. However, to account for the residual term $O(1/M)$, we substitute the concrete values into inequality (9) to verify whether the chosen parameters provide sufficient security, and adjust ϵ_0 accordingly if necessary. In particular, when $\epsilon = 0.5$, we require $T = \frac{M}{4} + 1$ to meet the desired security level. For $\epsilon > 0.5$ and sufficiently large M , setting $T = \frac{\epsilon M}{2}$ is already sufficient.

APPENDIX G

DETAILED CHEATER IDENTIFICATION MECHANISMS

A. General Cheater Identification

Note that the above cheater identification approach cannot apply to the normal (symmetric) MAC-based and triple-based approaches, as it's hard to identify an honest party without introducing other primitives. We thus design a general cheater identification mechanism using CP-NIZK proofs. The high-level idea is that, once an abortion occurs, we require each party to generate a zero-knowledge proof showing that it faithfully executed the protocol w.r.t. all the commitments (to its input and random tape) and signed messages.

To be more specific, after each party P_j picks its random tape tp_j , we require P_j to commit to its input $\vec{v}_j$ and random tape tp_j , and then broadcast the commitment (denoted by $\text{Com}_{\vec{v}_j}$, Com_{tp_j}) to the other parties. The hiding property of the commitment scheme ensures the privacy of the input and random tape, and the binding property guarantees that the corrupted party cannot forge other inputs or random tape after they are chosen. When an abortion occurs during the execution of the protocol, the party P_j first commits to all the signatures of its sent messages, and broadcasts the commitments,

Algorithm G.1: Blame

Input: The view of party P_j .

Auxiliary Input: A signature scheme $\text{SG} = (\text{Gen}, \text{Sign}, \text{Vrfy})$.

Algorithm:

- 1) Parse the last message in the cheater identification procedure as $(\text{corrupted}, \text{id}_i)$.
- 2) Run $\sigma_j := \text{SG.Sign}(\text{sk}_j; \text{id}_i)$, and sends σ_j to P_{3-i-j} .
- 3) After receiving σ_{3-i-j} , output $\text{cert}_j := (\text{id}_i, \sigma_j, \sigma_{3-i-j})$.

Algorithm G.2: Judge

Input: A certificate cert_j .

Auxiliary Input: A signature scheme $\text{SG} = (\text{Gen}, \text{Sign}, \text{Vrfy})$.

Algorithm:

- 1) Parse $\text{cert}_j := (\text{id}_i, \sigma_j, \sigma_{3-i-j})$.
- 2) For each $t \in \{j, 3 - i - j\}$, run $\text{SG.Vrfy}(\text{pk}_t; \text{id}_i, \sigma_t)$. If both checks are passed, output id_i , otherwise output $\perp$.

denoted by $\text{Com}_{\vec{v}_{j,i}}$ for $i \in \{j - 1, j + 1\}$ to the other parties, and meanwhile receives a commitment $\text{Com}_{\vec{v}_{i,j}}$ from each P_i . Then P_j is required to generate a zero-knowledge proof π_j showing that: all its sent messages with signatures (w.r.t. $\text{Com}_{\vec{v}_{j,i}}$) are computed faithfully according to the protocol specification using its inputs (w.r.t. $\text{Com}_{\vec{v}_j}$), random tape (Com_{tp_j}), and all the previously received messages with signatures (w.r.t. $\text{Com}_{\vec{v}_{i,j}}$).

Note that once the corrupted party ever cheated and led to the abortion, it cannot generate a valid proof showing its “honest” behaviors given the knowledge soundness property of the proof scheme. Thus, the party who cannot provide a valid proof will be identified as a cheater. Meanwhile, the zero-knowledge property ensures that no extra information is leaked about the honest parties’ private inputs.

We give a formal description of the above procedure in Protocol G.1. We claim this approach applies to any input-dependent online protocols, with no extra costs in the optimistic case (apart from those coming from commitments and signatures). Besides, almost all modern ZKP schemes are (or can be simply adapted to) CP-NIZK proof systems, and they typically feature different merits (such as $O(N)$ proving time, or $O(1)$ proof size and verifier time). Therefore, one can choose a CP-NIZK scheme according to the practical needs.

B. Blame and Judge Algorithm

We show the Blame and Judge algorithms in Algorithm G.1 and G.2. We give the following theorem.

Theorem G.1: Let f be a three-party functionality over $\mathbb{Z}_{2^k}$. For any protocol $\Pi_{\text{Cov}}^{\text{Abort}}$ securely computes f with covert security with abort and deterrence factor ϵ , $(\Pi_{\text{Cov}}^{\text{PV}}, \text{Blame}, \text{Judge})$ securely computes f in the presence of a covert adversary controlling one party, with public verifiability and deterrence factor ϵ .

Proof G.1: We prove the following three properties.

Simulatability. Simulatability follows from $\Pi_{\text{Cov}}^{\text{Abort}}$ as it securely computes f with covert security with abort.

Protocol G.1: $\Pi_{\text{Cov}}^{\text{PV}}$ (General Cheater Identification)

Input: Each party P_j where $j \in \{0, 1, 2\}$ holds inputs $v_j^{(i)} \in \mathbb{Z}_{2^k}$ where $i \in [d]$.

Auxiliary Input: A covertly secure protocol $\Pi_{\text{Cov}}^{\text{Abort}}$ with abort. The parties agree on a signature scheme $\text{SG} = (\text{Gen}, \text{Sign}, \text{Vrfy})$, a commitment scheme $\text{COM} = (\text{Gen}, \text{Commit}, \text{Open})$, and a zero-knowledge proof scheme $\text{ZK} = (\text{Gen}, \text{Prove}, \text{Vrfy})$. Let $\text{NMF}_j^{(\ell)}$ be the next message function of $\Pi_{\text{Cov}}^{\text{Abort}}$ that is executed by P_j in round ℓ , which takes as input the current view $\text{view}_j^{(\ell)}$, and outputs the messages that need to be sent.

1. Setup

- 1) Each party P_j runs $(\text{sk}_j, \text{pk}_j) \leftarrow \text{SG.Gen}(1^\kappa)$ to generate a signing key sk_j and a verification key pk_j , and broadcasts pk_j to the other two parties. The parties run $\text{pp} \leftarrow \text{COM.Gen}(1^\kappa)$ and $\text{crs} \leftarrow \text{ZK.Gen}(1^\kappa)$ to obtain the parameters pp and crs .

2. Commitment

- 1) Each party P_j samples a random tape denoted by tp_j . Let $\vec{v}_j$ be the vector of P_j 's inputs. P_j runs $\text{COM.Commit}(\text{pp}; \vec{v}_j)$ and $\text{COM.Commit}(\text{pp}; \text{tp}_j)$ to obtain commitments Com_{v_j} , Com_{tp_j} , and broadcast the two commitments to the other two parties.

3. Protocol Execution

The parties run protocol $\Pi_{\text{Cov}}^{\text{Abort}}$ with their individual inputs and random tape. For each round ℓ of protocol $\Pi_{\text{Cov}}^{\text{Abort}}$:

- 1) For each party P_j , let $\text{view}_j^{(\ell)}$ be the current view of it. P_j runs $(\text{msg}_{j,j-1}^{(\ell)}, \text{msg}_{j,j+1}^{(\ell)}) = \text{NMF}_j^{(\ell)}(\text{view}_j^{(\ell)})$ to compute the messages that should be sent to the other parties. If any message is abort, it broadcasts abort and goes to Step 4. Otherwise, it proceeds to the next step.
- 2) Each party P_j runs $\sigma_{j,i}^{(\ell)} := \text{SG.Sign}(\text{sk}_j; \ell || j || i || H(\text{msg}_{j,i}^{(\ell)}))$ to sign the messages, and sends the signature $\sigma_{j,i}^{(\ell)}$ along with the message to P_i . For each message $\text{msg}_{i,j}^{(\ell)}$ received by P_j , P_j verifies the signature $\sigma_{i,j}^{(\ell)}$ by running $\text{SG.Vrfy}(\text{pk}_i; \text{msg}_{i,j}^{(\ell)}, \sigma_{i,j}^{(\ell)})$.
- 3) If the signature $\sigma_{i,j}^{(\ell)}$ is invalid, P_j broadcasts (accuse, i) to the other parties. P_i then broadcasts another message $\hat{\text{msg}}_{i,j}^{(\ell)}$ with a signature $\hat{\sigma}_{i,j}^{(\ell)}$ to the other two parties, who then check if the new signature is valid w.r.t the new message. If it is still invalid, the other two parties broadcast $(\text{corrupted}, i)$ and go to Step 4. Otherwise, P_j uses the new message in the following execution.

4. Cheating Identification

Let ρ be the current number of rounds in the execution of the protocol $\Pi_{\text{Cov}}^{\text{Abort}}$.

- 1) Each party P_j runs $\pi_j := \text{ZK.Prove}(\text{crs}; s, w)$ to prove:
 - a) for each $\ell \in [\rho]$, there exist some secret messages $\text{msg}_{j,j-1}^\ell, \text{msg}_{j,j+1}^\ell$ that are correctly computed w.r.t. the function $\text{NMF}_j^{(\ell)}$ and the current view, i.e., $(\text{msg}_{j,j-1}^\ell, \text{msg}_{j,j+1}^\ell) = \text{NMF}_j^{(\ell)}(\text{view}_j^{(\ell)})$, and
 - b) for all $\ell \in [\rho]$ and $i \in \{j-1, j+1\}$, the signatures $\sigma_{j,i}^\ell, \sigma_{i,j}^\ell$ are correctly computed w.r.t. the signature scheme and the messages, i.e., $\sigma_{j,i}^\ell = \text{Sign}(\text{sk}_j; \ell || j || i || H(\text{msg}_{j,i}^\ell))$.
- 2) Each party P_j signs its proof π_j and gets a signature σ_{π_j} . If the last message sent by P_j before Step 4 is $(\text{corrupted}, i)$, P_j sends its proof and signature to the other party P_{3-i-j} , then outputs $(\text{corrupted}, i)$ and halts. Otherwise, it broadcasts the proof and the signature to the other parties. and proceeds to the next step.
- 3) If P_j doesn't receive a proof and signature from P_i , it outputs $(\text{corrupted}, i)$ and halts. Otherwise, it verifies the received proofs by running $\text{ZK.Vrfy}(\text{crs}; \pi_i)$ and $\text{SG.Vrfy}(\text{pk}_i; \pi_i, \sigma_{\pi_i})$. If any check on π_i or σ_{π_i} fails, it outputs $(\text{corrupted}, i)$ as well.

5. Output

- 1) If no $(\text{corrupted}, i)$ is output in previous steps, the parties output what the protocol $\Pi_{\text{Cov}}^{\text{Abort}}$ outputs.

Public Verifiability. We now show that if in a real execution the honest parties output $(\text{corrupted}, i)$, then the probability that the algorithm Judge outputs $\perp$ rather than P_i is negligible. First note that in $\Pi_{\text{Cov}}^{\text{PV}}$, the parties output $(\text{corrupted}, i)$ in the following three cases:

- 1) the party P_i fails to broadcast a message with a valid signature, or the commitment to its signatures is invalid,
- 2) the party P_i fails to broadcast a proof and a signature in the cheating identification phase, or
- 3) the proof or the signature of the proof from P_i is invalid.

In the first case, the parties proceed to generate zero-knowledge proofs to show their honest behaviors, and send their proofs to each other as certificates, then outputting $(\text{corrupted}, i)$. In the second and third cases, the parties directly output $(\text{corrupted}, i)$. Note that in the first two cases, the certificate of each honest party consists of two groups of proofs and signatures, which all come from the honest parties, while in the third case, the certificate consists of the proofs coming from all the parties.

The event that the algorithm Judge outputs $\perp$ indicates that

- In the first two cases, the check $\text{ZK.Vrfy}(\text{crs}; \pi_t)$ or $\text{SG.Vrfy}(\text{pk}_t; \pi_t, \sigma_{\pi_t})$ fails for some $t \in \{i-1, i+1\}$, or
- In the third case, the checks $\text{ZK.Vrfy}(\text{crs}; \pi_i)$ and $\text{SG.Vrfy}(\text{pk}_i; \pi_i, \sigma_{\pi_i})$ succeed, or the check on $\text{ZK.Vrfy}(\text{crs}; \pi_t)$ or $\text{SG.Vrfy}(\text{pk}_t; \pi_t, \sigma_{\pi_t})$ fails for some $t \in \{i-1, i+1\}$.

Due to the completeness of the zero-knowledge proof scheme and the correctness of the signature scheme, the proofs and signatures generated by the honest parties will always pass the checks. Due to the knowledge soundness of the zero-knowledge proof scheme and the unforgeability of the signature scheme, a malicious party cannot forge a proof or signature to pass the check except with negligible probability. Thus, we can conclude that in all three cases, the probability that the algorithm Judge outputs $\perp$ is negligible, and $\Pr[\text{Judge}(\text{cert}_j) = \text{id}_i] \geq 1 - \text{negl}(\kappa)$.

Defamation Freeness. If a corrupted party P_i controlled by $\mathcal{A}$ can successfully generate a certificate s.t. $\text{Judge}(\text{cert}_i) = \text{id}_j$ for $j \in \{i-1, i+1\}$, then it must hold that

- P_i successfully generated a valid proof π_i with a signature that passes the checks, and
- P_i forges an invalid proof or signature of P_j .

Given the knowledge soundness property of the zero-knowledge proof scheme, if any signed message sent by a corrupted party is inconsistent with the protocol specification, the probability that it can fake a proof to show the message is honestly computed is negligible. Thus, although a corrupted P_i can generate an invalid proof or signature and claim that they come from a “cheated” P_j , it cannot make Judge output P_j as it cannot generate another valid proof π_i to pass the check. Thus, we conclude that $\Pr[\text{Judge}(\text{cert}^*) = \text{id}_j \mid \text{cert}^* \leftarrow \mathcal{A}] \leq \text{negl}(\kappa)$. This concludes our proof. $\square$